> *From: https://chatgpt.com/c/6a8a1a24-1218-83e9-9f55-db7ec639b9a5*

you asked: go throgh

message time: 2026-08-23 03:20:26

go throgh the previous chat and responces how detailed- https://chatgpt.com/c/6a7f33eb-07f4-83ee-8229-383f952dcf93 . And continue further according to previous - For your goal, I recommend turning this into a structured **"2-Year Java + Spring Boot Product Company Interview Bible"** with:

Module 1: 150 Core Java questions + answers **Module 2:** 120 Spring/Spring Boot questions + internal working **Module 3:** 80 JPA/Hibernate questions **Module 4:** 75 REST/API questions **Module 5:** 75 Microservices questions **Module 6:** 50 Spring Security/JWT questions **Module 7:** 50 SQL questions + queries **Module 8:** 50 Multithreading questions **Module 9:** 50 Kafka + Redis questions **Module 10:** 50 Docker/AWS questions **Module 11:** 100 DSA problems specifically for 2-year Java SDE interviews **Module 12:** 50 LLD/system-design scenarios **Module 13:** 100 project/resume follow-up questions **Module 14:** Company-specific patterns for **Amazon, Google, Microsoft, Adobe, Uber, Walmart, Mastercard, Morgan Stanley, Cisco, Salesforce, Atlassian, Oracle, etc.**

Most importantly, for every question I'd structure it as:

Question → Short interview answer → Detailed explanation → Real project example → Follow-up question → Tricky interviewer counter-question → Best answer → Code/diagram where relevant

That format will train you to **answer follow-up questions instead of just memorizing definitions**..

Next Module: Spring & Spring Boot — 120 Questions

Now we move to the part that is **most important for your stated target: 2-year Spring Boot backend/product-company interviews.**

I will **not** make Module 2 merely:

> *What is @Autowired? What is @Component?*

Instead, we'll build it like an actual product-company interview.

Module 2 structure

Part A — Spring Core

Q1–30

- What is Spring?
- IoC
- DI

- BeanFactory
- ApplicationContext
- Bean lifecycle
- Constructor injection
- `@Autowired`
- `@Component`
- `@Service`
- `@Repository`
- `@Controller`
- `@Configuration`
- `@Bean`
- singleton/prototype/request/session scopes
- circular dependencies
- lazy initialization
- `@Primary`
- `@Qualifier`
- Spring proxies
- AOP

Part B — Spring Boot Internals

Q31–60

- Spring vs Spring Boot
- `@SpringBootApplication`
- auto-configuration
- starters
- conditional configuration
- `@ConditionalOnClass`
- `@ConditionalOnMissingBean`
- configuration properties
- profiles
- application startup
- embedded server
- Actuator
- custom configuration
- overriding Boot defaults
- debugging auto-configuration

Current 2026 interview material specifically emphasizes dependency injection, IoC, auto-configuration, starters, configuration/profiles, and understanding what happens internally when dependencies are added.

Part C — Spring MVC + REST

Q61–85

- DispatcherServlet
- Controller → Service → Repository
- `@RequestMapping`
- `@GetMapping`
- `@PostMapping`
- `@PathVariable`
- `@RequestParam`
- `@RequestBody`
- DTOs
- validation
- `@Valid`
- global exception handling
- `@ControllerAdvice`
- `ResponseEntity`
- HTTP status codes
- idempotency
- pagination
- API versioning

Part D — Production Spring Boot

Q86–105

- Transactions
- `@Transactional`
- propagation
- isolation
- rollback
- proxy behavior
- self-invocation
- transaction boundaries
- N+1
- connection pools

- caching
- logging
- Actuator
- health checks
- configuration/security mistakes

Part E — Actual Interview Scenarios

Q106–120

These will be the **most valuable**.

For example:

> *"Your API is suddenly taking 5 seconds instead of 200 ms. How do you investigate?"*
>
> *"You added @Transactional, but rollback isn't happening. Why?"*
>
> *"Two beans implement the same interface. What happens?"*
>
> *"Your application starts locally but fails in production. How do you debug it?"*
>
> *"Why does adding spring-boot-starter-data-jpa suddenly create database-related infrastructure?"*
>
> *"Your API returns 500 for invalid input. How would you redesign the exception handling?"*
>
> *"Your Spring Boot service has 10,000 DB queries for one API request. What's happening?"*

These are the questions that turn **memorization into interview performance**.

Recent 2-year interview guidance specifically recommends being able to build a small end-to-end feature—**controller → service → repository → DTO → validation → exception handling**—and explain the design decisions, rather than only knowing annotation definitions.

chatgpt response: Yes. I und

Yes. I understand what you want.

I checked the context available from the previous work and the structure you pasted. **We should continue from Module 2, not restart from the beginning.** Also, I'll keep the depth at the level you were aiming for: not just definitions, but **internal working + interview-ready answers + practical examples + follow-ups + traps**.

One limitation: I can't directly open that private `chatgpt.com/c/...` conversation URL from here. But the previous Module 2 structure you pasted, together with the earlier context, is enough to continue consistently.

I'll also use current Spring documentation where appropriate rather than relying on old Spring 4/5 explanations. The current Spring reference identifies IoC as a core Spring concept, and the current Boot documentation confirms how `@SpringBootApplication`, auto-configuration, component scanning, and DI work. (Home)

Module 2 — Spring & Spring Boot

Part A — Spring Core

Q1–Q30

We will start with **Q1**, and proceed sequentially.

Q1. What is Spring Framework?

⭐ Short interview answer

> *Spring is a Java application framework that provides infrastructure for building enterprise applications. Its core feature is the IoC container, which manages objects, their dependencies, and their lifecycle. Spring also provides modules for AOP, transactions, web development, data access, security integration, etc.*

For a 2-year interview, don't stop there.

The interviewer may immediately ask:

> *"What problem does Spring actually solve?"*

Your answer:

> *Without Spring, application classes often create and manage their dependencies themselves using new, which creates tight coupling. Spring moves object creation and dependency wiring to the IoC container. This gives us loose coupling, easier testing, centralized configuration, and infrastructure such as transactions and AOP.*

Spring's own documentation describes the IoC container as one of the fundamental parts of the framework. (Home)

Detailed explanation

Imagine a simple application without Spring.

```
java
 public class OrderService {


     private PaymentService paymentService =
             new PaymentService();
```

```
    public void placeOrder() {
        paymentService.processPayment();
    }
}
```

Here:

```
text
OrderService
     |
     | creates using new
     ↓
PaymentService
```

`OrderService` is directly responsible for creating `PaymentService`.

That creates **tight coupling**.

Now suppose:

```
java
PaymentService
```

changes to:

```
java
UPIPaymentService
```

or:

```
java
CardPaymentService
```

or you want to mock it during testing.

You have to modify `OrderService`.

With Spring

We can write:

```
java
@Service
public class OrderService {
```

```
    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }

    public void placeOrder() {
        paymentService.processPayment();
    }
}
```

And:

```java
@Service
public class PaymentService {

    public void processPayment() {
        System.out.println("Payment processed");
    }
}
```

Now `OrderService` doesn't create `PaymentService`.

Instead:

```text
                Spring IoC Container
                        |
              creates/manages beans
                        |
          +-------------+-------------+
          |                           |
          ↓                           ↓
   OrderService               PaymentService
          |
          | injected
          ↓
   PaymentService
```

Spring's DI model specifically allows objects to declare dependencies through constructors, factory methods, or properties, while the container supplies those dependencies. (Home)

What exactly does Spring manage?

This is an important interview point.

Spring can manage:

1. Object creation

```
java
 PaymentService paymentService = new PaymentService();
```

Spring can effectively perform this creation for a bean.

2. Dependency injection

```
java
 OrderService(PaymentService paymentService)
```

Spring resolves and supplies the dependency.

3. Bean lifecycle

Spring can manage initialization and destruction callbacks.

4. Configuration

For example:

```
properties
 payment.timeout=5000
```

5. Cross-cutting concerns

Such as:

```
text
 Logging
 Transactions
 Security
 Caching
 Auditing
```

often implemented using AOP/proxies.

6. Web infrastructure

Spring MVC provides components such as:

```
text
 DispatcherServlet
 Controller
 HandlerMapping
 HandlerAdapter
```

7. Database/transaction infrastructure

Spring integrates with:

```
text
 JDBC
 JPA
 Hibernate
 Transactions
 DataSource
```

Q1.1 What is the core of Spring?

Interview answer

> *The core of Spring is the IoC container. It manages Spring beans, their dependencies, configuration, and lifecycle.*

Think:

```
text
 Spring
   |
   +-- IoC Container
   |      |
   |      +-- Bean creation
   |      +-- Dependency Injection
   |      +-- Lifecycle
   |      +-- Configuration
   |
   +-- AOP
   |
   +-- Transactions
   |
```

```
   +-- Spring MVC
   |
   +-- Data Access
   |
   +-- Integration
```

The current Spring documentation explicitly describes the IoC container and AOP as fundamental Spring technologies. (Home)

Q1.2 What is a Spring Bean?

⭐ Interview answer

> *A Spring Bean is an object that is instantiated, configured, and managed by the Spring IoC container.*

Example:

```java
 @Service
 public class PaymentService {
 }
```

Spring discovers this class through component scanning and registers it as a bean.

You can then inject it:

```java
 @Service
 public class OrderService {

     private final PaymentService paymentService;

     public OrderService(PaymentService paymentService) {
         this.paymentService = paymentService;
     }
 }
```

The important distinction is:

```text
 Java Object
```

```
    ↓
created using new
```

versus:

```
text
Spring Bean
    ↓
created/managed by Spring IoC Container
```

Spring's bean definition contains metadata describing things such as the class, scope, dependencies, lifecycle callbacks, and configuration. (Home)

Q1.3 Is every Java object a Spring Bean?

✕No.

Example:

```
java
public class OrderService {
}
```

If you do:

```
java
OrderService service = new OrderService();
```

that object is **not automatically a Spring Bean**.

But:

```
java
@Service
public class OrderService {
}
```

can be discovered by Spring's component scanning and registered as a bean.

Or:

```
java
@Configuration
public class AppConfig {
```

```
    @Bean
    public OrderService orderService() {
        return new OrderService();
    }
}
```

Now the returned object is managed by Spring.

Spring Boot recommends component scanning for typical application components and constructor injection for dependencies. (Home)

Q1.4 What problem does Spring IoC solve?

Suppose you have:

```java
public class OrderService {

    private PaymentService paymentService =
            new PaymentService();
}
```

Problems:

```text
Tight coupling
      ↓
Harder testing
      ↓
Harder replacement
      ↓
Harder configuration
      ↓
Object lifecycle managed manually
```

With Spring:

```text
OrderService
      |
      | depends on
```

```
      ↓
 PaymentService
```

The dependency is supplied externally.

That's **Dependency Injection**.

Q1.5 Spring vs Spring Framework vs Spring Boot

This is a very common confusion.

Spring Framework

The underlying framework providing:

```
text
 IoC
 DI
 AOP
 MVC
 Transactions
 Data access
 etc.
```

Spring Boot

A project built on top of Spring Framework that simplifies application setup and configuration.

For example:

```
text
 Spring Framework
       ↓
 requires configuration/integration choices
       ↓
 Spring Boot
       ↓
 auto-configuration
 starters
 embedded server
 externalized configuration
 production features
```

We'll cover this deeply in **Q31 onward**.

Q1.6 Real project example

Suppose you're building an e-commerce application.

You have:

```
text
 OrderController
        ↓
 OrderService
        ↓
 PaymentService
        ↓
 PaymentRepository
        ↓
 Database
```

Without Spring:

```
java
 OrderController controller = new OrderController();


 OrderService orderService = new OrderService();


 PaymentService paymentService = new PaymentService();


 PaymentRepository repository =
         new PaymentRepository();
```

You would have to manually construct and connect the entire object graph.

With Spring:

```
text
                  Spring Container
                         |
         +---------------+--------------+
         |               |              |
         ↓               ↓              ↓
  Controller       Service         Repository
         |               |
         +---- injected-+
```

Spring builds the object graph for you.

This is one of the most important concepts to understand before moving into:

```
text
 IoC
 DI
 BeanFactory
 ApplicationContext
 Bean lifecycle
 @Autowired
 @Component
 @Configuration
 @Bean
```

Q1.7 Tricky interviewer question

Interviewer:

> *"Is Spring just a Dependency Injection framework?"*

❌Weak answer

> *Yes, Spring is mainly used for dependency injection.*

⭐ Better answer

> *No. Dependency Injection and IoC are at the core of Spring, but Spring is much broader. It provides infrastructure for AOP, transaction management, web development, data access, integration, testing, and more. Spring Boot builds on these capabilities and simplifies configuration and application startup.*

Q1.8 Another tricky question

Interviewer:

> *"Why not just use Java interfaces and dependency injection manually? Why do we need Spring?"*

⭐ Strong 2-year answer

> *We can implement DI manually, and for small applications that may be sufficient. Spring becomes valuable when the application has a large object graph and infrastructure requirements. It manages bean creation, dependency resolution, lifecycle, configuration, transactions, proxies, and other cross-cutting concerns consistently. It also reduces boilerplate and makes the application easier to configure and test.*

Q1.9 What happens internally when Spring starts?

This is where interview depth starts increasing.

Very simplified:

```
text
 Application starts
        ↓
 Create Spring ApplicationContext
        ↓
 Read configuration
        ↓
 Component scanning
        ↓
 Find bean definitions
        ↓
 Register BeanDefinitions
        ↓
 Resolve dependencies
        ↓
 Create beans
        ↓
 Apply BeanPostProcessors
        ↓
 Initialize beans
        ↓
 Create proxies where required
        ↓
 Application ready
```

Don't memorize this as an exact implementation sequence yet.

We will break it down throughout:

```
text
 Q2 → IoC
 Q3 → DI
 Q4 → BeanFactory
 Q5 → ApplicationContext
 Q6 → Bean lifecycle
```

```
Q7 → Constructor injection
...
```

Q1.10 One very important distinction

Spring does NOT magically inject objects everywhere.

Spring only manages objects that are part of its container.

For example:

```java
public class MyController {

    private final PaymentService service;

    public MyController() {
        this.service = new PaymentService();
    }
}
```

Spring cannot automatically manage the `PaymentService` created with:

```java
new PaymentService()
```

because **you created that object outside the container**.

Instead:

```java
@Service
public class PaymentService {
}
```

and:

```java
@RestController
public class MyController {

    private final PaymentService service;
```

```
    public MyController(PaymentService service) {
        this.service = service;
    }
}
```

Now Spring owns the dependency.

This distinction becomes extremely important when we discuss:

- `@Autowired`
- `@Transactional`
- AOP
- proxies
- self-invocation
- testing
- manually created objects

□ Interview-ready answer to memorize

If the interviewer simply asks:

> *"What is Spring?"*

Say:

> *"Spring is a Java application framework whose core is the IoC container. It manages application objects, called beans, and their dependencies through Dependency Injection. Besides IoC and DI, Spring provides capabilities such as AOP, transaction management, web development, and data access. The main benefit is loose coupling and centralized management of application components and infrastructure."*

That's a much stronger answer than:

> *"Spring is a framework used to develop Java applications."*

□ Q1 Interview Follow-up Tree

If interviewer asks:

```text
What is Spring?
      |
      +-- What is IoC?
      |      |
```

```
|      +-- What is DI?
|             |
|             +-- Constructor vs Setter?
|
+-- What is a Bean?
|
+-- Who creates Beans?
|
+-- What is ApplicationContext?
|
+-- What happens during startup?
|
+-- What is Bean lifecycle?
|
+-- What is @Autowired?
|
+-- Why constructor injection?
|
+-- How does Spring resolve dependencies?
|
+-- What happens if two beans have same type?
```

This is how you should prepare: don't learn Q1 as an isolated definition. Learn the entire chain.

Q2. What is IoC (Inversion of Control)?

⭐ Short interview answer

> *IoC means Inversion of Control. Instead of an application class controlling the creation and management of its dependencies, that responsibility is transferred to the Spring IoC container. Spring creates and manages beans and injects their dependencies.*

Without IoC

```java
public class OrderService {

    private PaymentService paymentService;

    public OrderService() {
        paymentService = new PaymentService();
```

```
    }
}
```

Here:

```
text
OrderService
     |
     | controls creation
     ↓
PaymentService
```

The control is inside `OrderService`.

With IoC

```java
@Service
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

Now:

```
text
OrderService
     |
     | declares dependency
     ↓
PaymentService

Spring Container
       |
       | creates + injects
       ↓
OrderService
```

The control has moved from the class to the container.

Hence:

> *Inversion of Control.*

Spring's documentation describes DI as a specialized form of IoC: the object declares its dependencies, while the container supplies them. (Home)

Q2.1 IoC vs DI

This is frequently asked.

IoC

A **principle**.

> *Control over object creation/dependency management is transferred to an external container/framework.*

DI

A **technique/pattern used to implement IoC**.

```text
 IoC
  |
  +-- Dependency Injection
  |       |
  |       +-- Constructor Injection
  |       +-- Setter Injection
  |       +-- Field Injection
  |
  +-- Other approaches
```

So:

> *DI is one way of achieving IoC.*

Don't say:

> *"IoC and DI are exactly the same thing."*

They are closely related, but conceptually not identical.

Q2.2 Real-world analogy

Imagine a restaurant.

Without IoC

The chef has to:

```
text
 buy vegetables
 buy meat
 buy spices
 hire staff
 manage suppliers
 cook food
```

The chef controls everything.

With IoC

The restaurant manager handles:

```
text
 suppliers
 ingredients
 staff
 inventory
```

The chef simply declares what is required:

```
text
 Need vegetables
 Need meat
 Need spices
```

The manager provides them.

Similarly:

```
text
 Application class
       |
       | declares dependency
       ↓
 Spring Container
       |
```

```
      | supplies dependency
      ↓
Application class
```

Q2.3 Interviewer counter-question

> *"Where exactly is the inversion?"*

⭐ Best answer

> *Normally, a class would control the creation or lookup of its dependencies. With Spring, the class only declares what it needs, and the IoC container controls creation, configuration, and injection. Therefore the control of object creation is inverted from the application code to the framework.*

Excellent answer for a 2-year interview.

Q2.4 Does IoC mean Spring creates everything using reflection?

❌Don't answer simply "yes."

Spring can use reflection and other mechanisms to instantiate and configure beans, but IoC itself is the **design principle**, not a synonym for reflection.

For example:

```text
IoC
 ↓
container controls object management

Reflection
 ↓
one mechanism Spring may use internally
```

This distinction matters in deeper interviews.

Q2.5 How does IoC help testing?

Suppose:

```java
@Service
public class OrderService {
```

```
    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

In production:

```
text
 OrderService
       ↓
 RealPaymentService
```

In unit testing:

```
text
 OrderService
       ↓
 MockPaymentService
```

Example:

```
java
 PaymentService mockPaymentService = mock(PaymentService.class);

 OrderService orderService =
         new OrderService(mockPaymentService);
```

You don't need to modify `OrderService`.

That's one major benefit of dependency injection.

Spring's documentation explicitly notes that DI improves decoupling and makes classes easier to test, particularly when dependencies are interfaces or abstract types. (Home)

□ Q2 Interview Scenario

Interviewer:

> *"I have 20 services in my application. Why should I care about IoC?"*

Strong answer:

> *IoC prevents each service from managing the construction and lifecycle of its dependencies. The container maintains the object graph and resolves dependencies centrally. This reduces coupling, improves testability, allows implementations to be replaced more easily, and provides a foundation for Spring features such as lifecycle management, AOP and transactions.*

Q3. What is Dependency Injection?

⭐ Short interview answer

> *Dependency Injection is the process where an object's dependencies are provided to it from outside rather than the object creating or looking them up itself. In Spring, the IoC container creates the beans and injects their dependencies.*

Example

Suppose:

```
java
 public interface PaymentService {
     void pay();
 }
```

Implementation:

```
java
 @Service
 public class CardPaymentService
         implements PaymentService {

     @Override
     public void pay() {
         System.out.println("Card payment");
     }
 }
```

Consumer:

```
java
 @Service
 public class OrderService {

     private final PaymentService paymentService;
```

```
    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

Here:

```
text
OrderService
      |
      | needs
      ↓
PaymentService

Spring
  |
  | finds implementation
  ↓
CardPaymentService
  |
  | injects
  ↓
OrderService
```

Three common forms of DI

1. Constructor Injection ⭐⭐⭐

```
java
@Service
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

Recommended for required dependencies.

Spring Boot's current documentation recommends constructor injection. (Home)

2. Setter Injection

```java
 @Service
 public class OrderService {

     private PaymentService paymentService;

     @Autowired
     public void setPaymentService(
             PaymentService paymentService) {
         this.paymentService = paymentService;
     }
 }
```

Useful when a dependency is optional or needs to be changed after construction in specific designs.

3. Field Injection

```java
 @Service
 public class OrderService {

     @Autowired
     private PaymentService paymentService;
 }
```

It works, but constructor injection is generally preferred.

We'll discuss **why** in Q7.

Q3.1 What exactly is a dependency?

If:

```java
 class OrderService {

     private PaymentService paymentService;
 }
```

then:

```text
 OrderService
       ↓
 depends on
       ↓
 PaymentService
```

`PaymentService` is a dependency of `OrderService`.

A dependency is simply another object/class that a class needs to perform its work.

Q3.2 Dependency Injection vs Dependency Inversion Principle

Very important.

These are **not the same thing**.

Dependency Injection

A mechanism/design technique:

```text
 Give object its dependencies from outside.
```

Dependency Inversion Principle

SOLID principle:

> *High-level modules should not depend directly on low-level modules; both should depend on abstractions.*

Example:

```java
 class OrderService {


     private final PaymentService paymentService;
 }
```

where:

```java
 interface PaymentService
```

is the abstraction.

Then:

```
text
 OrderService
      |
      ↓
 PaymentService interface
      ↑
      |
 CardPaymentService
```

Q3.3 Tricky interviewer question

> *"If I use @Autowired, am I automatically following Dependency Inversion?"*

Answer:

> *No. @Autowired provides dependency injection, but Dependency Inversion is about the dependency relationship between modules. I can technically inject a concrete implementation using @Autowired and still have poor abstraction. DI helps implement DIP, but they are not synonymous.*

That's exactly the type of distinction product-company interviewers like.

Q3.4 Why is constructor injection preferred?

We'll cover this deeply in **Q7**, but the short answer is:

```
text
 Constructor injection
         ↓
 required dependencies explicit
         ↓
 object cannot easily exist in invalid state
         ↓
 final fields possible
         ↓
 easier unit testing
         ↓
 better immutability
```

Q3.5 What happens if Spring cannot find a dependency?

Suppose:

```java
@Service
public class OrderService {

    public OrderService(PaymentService paymentService) {
    }
}
```

But there is no bean implementing:

```java
PaymentService
```

Spring cannot satisfy the dependency.

Application startup generally fails with an exception such as:

```text
NoSuchBeanDefinitionException
```

or a related dependency-resolution failure, depending on the situation.

The important interview answer is:

> *The application context cannot successfully create the dependent bean because the required dependency cannot be resolved.*

Q3.6 What if there are two implementations?

Suppose:

```java
@Service
class CardPaymentService
        implements PaymentService {
}
```

and:

```java
@Service
```

```
class UPIPaymentService
        implements PaymentService {
}
```

Then:

```java
@Service
class OrderService {

    public OrderService(PaymentService paymentService) {
    }
}
```

Spring sees:

```text
PaymentService
      |
      +-- CardPaymentService
      |
      +-- UPIPaymentService
```

Which one should it inject?

Ambiguous.

Typically you'll get an error such as:

```text
NoUniqueBeanDefinitionException
```

Solutions include:

```java
@Primary
```

or:

```java
@Qualifier("cardPaymentService")
```

We'll cover both later.

□ Very important interview distinction

You should now be able to explain:

```
text
 Spring
  ↓
 IoC
  ↓
 DI
  ↓
 Bean
  ↓
 Container
```

But these aren't interchangeable words.

Spring

Framework.

IoC

Design principle.

DI

Technique for achieving IoC.

Bean

Object managed by Spring.

IoC Container

Component responsible for managing those beans.

Q4. What is BeanFactory?

⭐ Short interview answer

> *BeanFactory is Spring's basic IoC container interface. It provides mechanisms for creating and retrieving beans and managing their dependencies. ApplicationContext builds on BeanFactory and provides additional enterprise/application-level features.*

Spring's current documentation describes `BeanFactory` as an advanced configuration mechanism capable of managing objects, while `ApplicationContext` represents the more feature-rich IoC container used in typical applications. (Home)

Think of it like this

```text
 BeanFactory
      |
      | basic IoC container
      |
      +-- Bean creation
      +-- Dependency injection
      +-- Bean lookup
      +-- Bean lifecycle support
```

Then:

```text
 ApplicationContext
         |
         | extends BeanFactory capabilities
         |
         +-- IoC
         +-- DI
         +-- Events
         +-- Internationalization
         +-- Resource loading
         +-- Environment/profiles
         +-- More application infrastructure
```

Q4.1 Is ApplicationContext a BeanFactory?

Conceptually:

> *ApplicationContext extends BeanFactory and provides additional functionality on top of it.*

So:

```text
 ApplicationContext
         ↓
 BeanFactory
```

Q4.2 Why don't we normally use BeanFactory directly?

Because most modern Spring applications benefit from the richer capabilities of `ApplicationContext`.

Typical Spring Boot application:

```java
 SpringApplication.run(Application.class, args);
```

creates/configures an application context for the application.

You normally don't write:

```java
 BeanFactory factory = ...
```

for your everyday Spring Boot application.

Q4.3 Interviewer asks:

> *"Which one is better: BeanFactory or ApplicationContext?"*

Don't say:

> *ApplicationContext is better.*

Say:

> *They serve different levels of functionality. BeanFactory provides the fundamental IoC container functionality, while ApplicationContext extends it with additional application-oriented features. In typical Spring and Spring Boot applications, ApplicationContext is preferred.*

Q5. What is ApplicationContext?

⭐ Short interview answer

> *ApplicationContext is Spring's central application container. It manages bean creation, dependency injection, lifecycle, configuration, and also provides additional features such as event publishing, resource loading, internationalization, and environment/profile support.*

The Spring reference describes `ApplicationContext` as the IoC container responsible for instantiating, configuring, and assembling beans from configuration metadata. (Home)

High-level architecture

```text
                  ApplicationContext
                         |
```

```
        +--------------+---------------+
        |              |               |
        ↓              ↓               ↓
    BeanFactory     Environment      Events
        |
        ↓
 Bean Management
        |
 +------+------+------+
 |      |      |      |
 ↓      ↓      ↓      ↓
Bean1  Bean2  Bean3  Bean4
```

Q5.1 What happens when ApplicationContext starts?

At a high level:

```text
 Application starts
        ↓
 Create ApplicationContext
        ↓
 Read configuration
        ↓
 Discover/register BeanDefinitions
        ↓
 Resolve dependencies
        ↓
 Instantiate required beans
        ↓
 Initialize beans
        ↓
 Apply post-processors/proxies where required
        ↓
 Context becomes ready
```

The exact startup internals are more nuanced, and we'll go deep into them later.

Q5.2 Where are Spring Beans stored?

At a conceptual level, the container maintains bean definitions and singleton instances in its bean factory infrastructure.

For interview purposes:

```
text
 ApplicationContext
        ↓
 BeanFactory infrastructure
        ↓
 Bean definitions
        ↓
 Singleton bean instances
```

Don't oversimplify it to:

> *"All beans are stored in ApplicationContext."*

The actual implementation uses bean-factory infrastructure and caches.

That distinction becomes useful when discussing singleton scope and bean lifecycle.

Q5.3 What is the difference between `getBean()` and dependency injection?

You can manually ask the container:

```
java
 ApplicationContext context = ...;

 PaymentService service =
         context.getBean(PaymentService.class);
```

That's **service locator style lookup**.

With DI:

```
java
 public OrderService(PaymentService service) {
     this.service = service;
 }
```

the dependency is supplied automatically.

Preferred:

```text
 DI
  ↓
 dependencies explicit
  ↓
 better decoupling/testing
```

versus:

```text
 getBean()
  ↓
 class actively asks container
  ↓
 more coupling to Spring
```

This is a very good follow-up question if an interviewer wants to test whether you truly understand DI.

Q6. What is the Spring Bean Lifecycle?

⭐ Short interview answer

> *The Spring bean lifecycle describes the stages a bean goes through from creation to destruction. Spring instantiates the bean, injects dependencies, applies bean post-processors, performs initialization callbacks, makes the bean available for use, and eventually invokes destruction callbacks when the container shuts down, depending on the bean scope.*

High-level lifecycle

For a typical singleton bean:

```text
 Bean Definition
       ↓
 Instantiate bean
       ↓
 Populate dependencies
       ↓
 Aware callbacks
       ↓
 BeanPostProcessor
    before initialization
```

```
      ↓
@PostConstruct
      ↓
InitializingBean
      ↓
custom init method
      ↓
BeanPostProcessor
   after initialization
      ↓
Bean ready
      ↓
Application uses bean
      ↓
Context shutdown
      ↓
@PreDestroy
      ↓
DisposableBean
      ↓
custom destroy method
```

This is a **high-value interview topic**.

Q6.1 Why is BeanPostProcessor important?

Because Spring uses post-processors as extension points around bean creation/initialization.

Conceptually:

```
text
create object
     ↓
post-process
     ↓
initialize
     ↓
post-process
     ↓
final bean
```

They are also important for understanding how infrastructure such as proxies can be introduced.

That leads directly to later questions about:

```text
 @Transactional
 @Async
 @Cacheable
 AOP
 Spring Security
```

Q6.2 What happens with `@PostConstruct`?

Example:

```java
 @Service
 public class PaymentService {

     @PostConstruct
     public void init() {
         System.out.println("Initialized");
     }
 }
```

Spring invokes the initialization callback after dependency injection and before the bean is ready for normal use.

Q6.3 What happens at shutdown?

For managed singleton beans, Spring can invoke destruction callbacks.

Example:

```java
 @PreDestroy
 public void cleanup() {
     System.out.println("Cleanup");
 }
```

Conceptually:

```text
Application running
        ↓
Bean active
        ↓
Application shutdown
        ↓
Destroy callbacks
        ↓
Bean destroyed
```

Important scope caveat: Spring does not manage the complete lifecycle of prototype beans after handing them to the client, so destruction callbacks for prototype beans are not automatically invoked by the container. (Home)

Q6.4 Interviewer trap

> *"Does Spring call @PreDestroy for every bean?"*

✕Wrong

> *Yes.*

⭐ Correct

> *It depends on the bean scope and lifecycle management. Singleton beans are fully managed by the container, while prototype beans are instantiated and configured by Spring but their destruction lifecycle is generally the responsibility of the client.*

That is a much stronger answer. (Home)

Q7. Why is Constructor Injection preferred over Field Injection?

⭐ Short interview answer

> *Constructor injection makes required dependencies explicit, supports immutable final fields, makes the object easier to test, and prevents creating an object without its required dependencies. It also exposes circular dependencies more clearly.*

Spring Boot's current documentation recommends constructor injection for wiring dependencies. (Home)

Field injection

```java
@Service
```

```
public class OrderService {

    @Autowired
    private PaymentService paymentService;
}
```

Problems:

1. Dependency is hidden

Looking at:

```java
new OrderService()
```

doesn't tell you that `PaymentService` is required.

2. Harder unit testing

You can't naturally do:

```java
OrderService service =
        new OrderService(mockPaymentService);
```

because there is no constructor.

3. Can't naturally use `final`

```java
private final PaymentService paymentService;
```

is a much stronger design.

Constructor injection

```java
@Service
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
```

```
    }
}
```

Now the class clearly communicates:

> *"I cannot function without PaymentService."*

Q7.1 What if a class has 10 constructor dependencies?

Excellent interview follow-up.

If you have:

```java
public OrderService(
        A a,
        B b,
        C c,
        D d,
        E e,
        F f,
        G g,
        H h,
        I i,
        J j) {
}
```

Don't conclude:

> *Constructor injection is bad.*

Instead ask:

> *Why does this class have so many responsibilities/dependencies?*

It may indicate a violation of:

```text
Single Responsibility Principle
```

or a class that is becoming a "God service."

Q7.2 Can Spring inject through a constructor without `@Autowired`?

Yes, in the common case where the class has a single constructor.

Example:

```java
@Service
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

You don't necessarily need:

```java
@Autowired
```

on that single constructor.

This is another good practical detail to know.

Q8. What is `@Autowired`?

⭐ Short interview answer

> *@Autowired tells Spring to resolve and inject a suitable bean into an injection point, such as a constructor, setter, or field. Spring resolves the dependency primarily by type, with mechanisms such as qualifiers or primary beans used to resolve ambiguity.*

Example:

```java
@Service
public class OrderService {

    private final PaymentService paymentService;

    @Autowired
    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
```

```
    }
}
```

How does Spring resolve it?

Suppose:

```
java
PaymentService
```

is required.

Spring searches its container for compatible beans.

```
text
Required:
PaymentService
       ↓
Container searches
       ↓
PaymentService beans
       ↓
one candidate?
       |
       +-- YES → inject
       |
       +-- NO → dependency error
       |
       +-- multiple → ambiguity
```

Multiple candidates can be resolved using:

```
java
@Primary
```

or:

```
java
@Qualifier
```

Q8.1 What happens if there are two beans?

```java
 @Service
 class CardPaymentService
         implements PaymentService {
 }
```

```java
 @Service
 class UpiPaymentService
         implements PaymentService {
 }
```

Then:

```java
 public OrderService(PaymentService paymentService) {
 }
```

Spring has two candidates.

Usually:

```text
 NoUniqueBeanDefinitionException
```

Q8.2 How do you solve it?

Option 1 — `@Primary`

```java
 @Primary
 @Service
 public class CardPaymentService
         implements PaymentService {
 }
```

Now CardPaymentService becomes the default candidate when multiple beans match.

Option 2 — `@Qualifier`

```java
 public OrderService(
     @Qualifier("upiPaymentService")
```

```
    PaymentService paymentService) {


    this.paymentService = paymentService;
}
```

Now you're explicitly selecting the bean.

Q9. What is `@Component`?

⭐ Short interview answer

> *@Component is a stereotype annotation that marks a class as a Spring-managed component. During component scanning, Spring detects the class and registers it as a bean.*

Example:

```
java
@Component
public class EmailValidator {
}
```

Conceptually:

```
text
@Component
      ↓
Component scanning
      ↓
BeanDefinition
      ↓
Spring container
      ↓
Spring Bean
```

Q9.1 `@Component` vs `@Service`

`@Service` is a specialized stereotype used for service-layer classes.

```
java
@Service
public class OrderService {
}
```

versus:

```java
@Component
public class EmailValidator {
}
```

Both can be detected as Spring components.

Semantically:

```text
@Component
   |
   +-- @Service
   +-- @Repository
   +-- @Controller
```

The specialized annotations communicate intent.

Q10. What is `@Service`?

⭐ Interview answer

> *@Service is a specialization of @Component used to indicate that a class belongs to the service/business-logic layer. It is detected during component scanning and registered as a Spring bean.*

Example:

```java
@Service
public class OrderService {

    public void placeOrder() {
        // business logic
    }
}
```

Typical architecture:

```text
Controller
    ↓
```

```
@Service
    ↓
@Repository
    ↓
Database
```

Q11. What is `@Repository`?

⭐ Interview answer

> *@Repository is a specialization of @Component used for persistence/data-access components. It communicates the role of the class and participates in Spring's exception translation mechanism for supported persistence technologies.*

Example:

```java
@Repository
public class OrderRepository {
}
```

Important:

Don't say:

> *"@Repository is only used to connect to the database."*

It indicates a **data-access component**, and Spring can apply persistence exception translation.

Q12. What is `@Controller`?

⭐ Interview answer

> *@Controller marks a class as a Spring MVC controller responsible for handling web requests and returning a view or participating in MVC request processing.*

Example:

```java
@Controller
public class OrderController {

    @GetMapping("/orders")
    public String orders() {
        return "orders";
```

```
    }
}
```

For REST APIs, you'll commonly use:

```java
@RestController
```

which we'll discuss in Part C.

Q13. What is `@Configuration`?

⭐ Short answer

> *@Configuration marks a class as a source of bean definitions and configuration for the Spring container.*

Example:

```java
@Configuration
public class AppConfig {

    @Bean
    public PaymentService paymentService() {
        return new PaymentService();
    }
}
```

Spring documentation identifies `@Configuration` classes and `@Bean` methods as central artifacts of Java-based configuration. (Home)

Q14. What is `@Bean`?

⭐ Short answer

> *@Bean tells Spring that the object returned by a method should be registered and managed as a Spring bean. It is useful when you need to configure or register objects that you don't directly control with component annotations.*

Example:

```java
@Configuration
public class AppConfig {
```

```
    @Bean
    public ObjectMapper objectMapper() {
        return new ObjectMapper();
    }
}
```

Now:

```
text
@Bean method
     ↓
Object created
     ↓
Spring container
     ↓
Object managed as bean
```

Q14.1 `@Component` vs `@Bean`

This is **very important**.

`@Component`

Put annotation on the class:

```
java
@Component
public class EmailService {
}
```

Spring discovers it through component scanning.

`@Bean`

Put annotation on a method:

```
java
@Configuration
public class Config {

    @Bean
    public EmailService emailService() {
```

```
        return new EmailService();
    }
}
```

You explicitly tell Spring how to create/register the object.

When would you use `@Bean`?

Especially when:

```
text
Third-party library
        ↓
You cannot modify its class
        ↓
Need to register/configure it
        ↓
@Bean
```

Example:

```
java
@Bean
public ObjectMapper objectMapper() {
    return new ObjectMapper();
}
```

Q15. What is Singleton Scope in Spring?

⭐ Short interview answer

> *Singleton is the default Spring bean scope. It means one bean instance is created per bean definition within a particular Spring IoC container.*

Current Spring documentation emphasizes an important distinction: Spring's singleton is **per container and per bean**, not the same as the classic GoF singleton pattern. (Home)

Example

```
java
@Service
public class PaymentService {
}
```

If the bean is singleton-scoped:

```
text
 Spring Container
       |
       ↓
 PaymentService instance #1
```

Multiple injections:

```
text
 OrderService ──┐
                ↓
           PaymentService
                ↑
                |
 RefundService ─┘
```

Both can reference the same singleton bean instance.

Q15.1 Is Spring Singleton the same as Java Singleton?

✗No.

Classic singleton:

```
java
 private static final Instance INSTANCE = ...
```

tries to enforce one instance according to its class-loading/runtime design.

Spring singleton means:

> *One instance of that bean definition per Spring container.*

You can have:

```
text
 Container A
    ↓
 PaymentService #1


 Container B
```

```
    ↓
PaymentService #2
```

Therefore:

```
text
Spring singleton ≠ GoF singleton
```

Q16. What is Prototype Scope?

⭐ Short answer

> *Prototype scope causes Spring to create a new instance each time the bean is requested from the container.*

Example:

```
java
@Component
@Scope("prototype")
public class ReportBuilder {
}
```

Then conceptually:

```
java
context.getBean(ReportBuilder.class);
```

→ instance 1

Another:

```
java
context.getBean(ReportBuilder.class);
```

→ instance 2

```
text
getBean()
    ↓
new instance


getBean()
```

```
    ↓
another new instance
```

Spring's documentation states that prototype beans result in a new instance whenever the bean is requested/resolved. (Home)

Q16.1 Tricky question

> *"If I inject a prototype bean into a singleton bean, will I get a new prototype every time?"*

✗No.

This is a classic Spring interview trap.

Suppose:

```java
@Component
@Scope("prototype")
class ReportBuilder {
}
```

and:

```java
@Service
class ReportService {

    private final ReportBuilder builder;

    public ReportService(ReportBuilder builder) {
        this.builder = builder;
    }
}
```

`ReportService` is singleton.

The prototype dependency is resolved when the singleton is created.

So:

```text
Singleton ReportService
        |
```

```
        ↓
Prototype ReportBuilder #1
```

Repeated method calls on the singleton don't automatically produce:

```
text
#2
#3
#4
```

The same injected reference remains.

Spring's documentation explicitly calls this out: dependencies are resolved when the singleton is instantiated. (Home)

Q17. What are the other Spring bean scopes?

Spring supports:

```
text
singleton
prototype
request
session
application
websocket
```

The web-related scopes require a web-aware application context. (Home)

Summary

Scope	Meaning
singleton	One instance per bean definition/container
prototype	New instance when requested
request	One instance per HTTP request
session	One instance per HTTP session
application	One instance per ServletContext
websocket	One instance per WebSocket session

Q18. What is `@Lazy`?

⭐ Short answer

> *@Lazy tells Spring to delay bean initialization until the bean is actually needed instead of eagerly creating it during application-context startup.*

Example:

```
java
 @Lazy
 @Service
 public class HeavyReportService {
 }
```

Conceptually:

Without lazy:

```
text
 Application startup
       ↓
 Create HeavyReportService
```

With lazy:

```
text
 Application startup
       ↓
 Bean not initialized yet
       ↓
 First required access
       ↓
 Create bean
```

Q18.1 Why use lazy initialization?

Useful when:

```
text
 expensive initialization
 rarely used component
 startup optimization
```

But don't blindly put `@Lazy` everywhere.

It can simply move an initialization failure from:

```
text
 startup
```

to:

```
text
 first request/use
```

That can be undesirable in production.

Q19. What is `@Primary`?

⭐ Short answer

> *@Primary marks one bean as the preferred candidate when multiple beans match a dependency by type.*

Example:

```
java
 @Service
 @Primary
 public class CardPaymentService
         implements PaymentService {
 }
```

Another:

```
java
 @Service
 public class UpiPaymentService
         implements PaymentService {
 }
```

Then:

```
java
 public OrderService(PaymentService paymentService) {
 }
```

Spring chooses the primary bean.

Q19.1 `@Primary` vs `@Qualifier`

`@Primary`

Defines a **default/preferred choice**.

```
java
 @Primary
 @Service
 class CardPaymentService
```

`@Qualifier`

Defines an **explicit choice**.

```
java
 public OrderService(
     @Qualifier("upiPaymentService")
     PaymentService paymentService) {
 }
```

Think:

```
text
 @Primary
     ↓
 default choice


 @Qualifier
     ↓
 specific choice
```

Q20. What is `@Qualifier`?

⭐ Interview answer

> *@Qualifier is used to disambiguate between multiple beans of the same type by specifying which bean should be injected.*

Example:

```
java
 @Service("cardPayment")
 class CardPaymentService
```

```
        implements PaymentService {
}
```

```java
@Service("upiPayment")
class UpiPaymentService
        implements PaymentService {
}
```

Then:

```java
public OrderService(
    @Qualifier("upiPayment")
    PaymentService paymentService) {
}
```

Now:

```text
PaymentService
     |
     +-- cardPayment
     |
     +-- upiPayment
              ↑
              |
          @Qualifier
```

Q21. What is a circular dependency?

⭐ Short interview answer

> *A circular dependency occurs when Bean A depends on Bean B while Bean B directly or indirectly depends on Bean A. Constructor-based circular dependencies cannot be resolved because neither object can be constructed without the other.*

Example:

```java
@Service
class A {
```

```
    A(B b) {}
}

java
@Service
class B {

    B(A a) {}
}
```

Graph:

```
text
A
↓
B
↓
A
↓
B
...
```

Spring cannot construct either one first.

With constructor injection, this results in a failure such as:

```
text
BeanCurrentlyInCreationException
```

Spring's current documentation explicitly notes that constructor injection can create an unresolvable circular dependency and result in `BeanCurrentlyInCreationException`. (Home)

Q21.1 How do you fix circular dependency?

Best answer:

> *The preferred solution is to redesign the dependency graph rather than simply work around the cycle.*

For example:

```
text
A → B
B → A
```

might indicate responsibilities are poorly separated.

Possible redesign:

```text
A → C
B → C
```

or introduce an abstraction/application service.

You can sometimes break cycles through techniques such as setter injection or `@Lazy`, depending on the situation, but for production design:

> *Prefer removing the circular dependency.*

Q22. What is Spring Proxy?

⭐ Short interview answer

> *A Spring proxy is an object that wraps or stands in front of a target bean so that Spring can intercept method calls and apply additional behavior such as transactions, caching, security, or AOP advice.*

Conceptually:

```text
Caller
  |
  ↓
Proxy
  |
  | interception
  ↓
Target Bean
```

For example:

```java
@Transactional
public void transferMoney() {
}
```

Conceptually:

```text
 Controller
     ↓
 Transaction Proxy
     ↓
 begin transaction
     ↓
 transferMoney()
     ↓
 commit / rollback
```

This concept will become **extremely important** in:

```text
 @Transactional
 AOP
 self-invocation
 @Async
 @Cacheable
 Spring Security
```

Q22.1 Why does Spring need proxies?

Because Spring wants to add behavior around your method without forcing you to manually write that infrastructure.

For example:

```java
 @Transactional
```

You write:

```java
 transferMoney();
```

But Spring can effectively arrange:

```text
 begin transaction
       ↓
 execute method
```

```
      ↓
success?
  /       \
yes       no
 ↓         ↓
commit   rollback
```

That interception is commonly implemented using proxies.

Q23. What is AOP?

⭐ Short answer

> *AOP, or Aspect-Oriented Programming, is a programming approach for separating cross-cutting concerns such as logging, security, transactions, and auditing from the core business logic.*

Example:

Without AOP:

```java
public void transfer() {

    log();

    checkSecurity();

    beginTransaction();

    // business logic

    commitTransaction();

    audit();
}
```

Business logic becomes polluted.

With AOP:

```java
@Loggable
```

```
@Secured
@Transactional
public void transfer() {

    // only business logic
}
```

Conceptually:

```
text
Caller
  ↓
Proxy
  ↓
Security
  ↓
Transaction
  ↓
Logging
  ↓
Target method
```

Q24. What are cross-cutting concerns?

These are concerns that affect multiple parts of the application.

Examples:

```
text
Logging
Security
Transactions
Auditing
Caching
Performance monitoring
```

Without AOP:

```
text
Service A → logging code
Service B → logging code
```

```
Service C → logging code
Service D → logging code
```

With AOP:

```
text
               Logging Aspect
                     |
                     ↓
Service A ←--------+
Service B ←--------+
Service C ←--------+
Service D ←--------+
```

Q25. What is the difference between a Spring Bean and a normal Java object?

⭐ Interview answer

A normal Java object:

```
java
PaymentService service =
        new PaymentService();
```

is created and managed by your code.

A Spring Bean:

```
java
@Service
class PaymentService {
}
```

is registered with and managed by the Spring container.

Spring can manage:

```
text
creation
dependency injection
configuration
lifecycle
```

```
scope
post-processing
proxying
```

Q26. Can we create a Spring Bean manually?

Yes.

Using:

```java
@Bean
public PaymentService paymentService() {
    return new PaymentService();
}
```

or programmatic registration mechanisms.

The important point:

```java
new PaymentService()
```

by itself does not make it a Spring Bean.

Q27. What is component scanning?

⭐ Short answer

> *Component scanning is the mechanism through which Spring searches configured packages for classes annotated with component stereotypes such as @Component, @Service, @Repository, and @Controller, and registers them as beans.*

Example:

```java
@Service
public class OrderService {
}
```

Spring scans the package:

```text
com.example
```

```
    |
    +-- controller
    +-- service
    +-- repository
```

and discovers:

```text
OrderService
```

Then registers its bean definition.

Q28. Why does package structure matter in Spring Boot?

Because component scanning has a base package.

Typical:

```text
com.company.app
    |
    +-- Application.java
    |
    +-- controller
    +-- service
    +-- repository
```

If:

```java
@SpringBootApplication
public class Application {
}
```

is located at:

```text
com.company.app
```

component scanning normally covers that package and its subpackages.

This is why putting the main application class in an appropriate top-level package is an important Spring Boot convention. `@SpringBootApplication` includes component scanning. (Home)

Q29. `@Component` vs `@Service` vs `@Repository` vs `@Controller`

Interview answer

All are Spring stereotype annotations used to identify components, but they communicate different architectural roles.

```text
 @Component
    |
    +-- generic component

 @Service
    |
    +-- business/service layer

 @Repository
    |
    +-- persistence/data-access layer

 @Controller
    |
    +-- Spring MVC web/controller layer
```

Don't answer:

> *"They are completely different mechanisms."*

They participate in the component model but carry different semantics and, for `@Repository`, persistence exception translation support.

Q30. What happens internally when Spring discovers `@Component`?

This is an excellent **Part A closing question**.

Suppose:

```java
 @Service
 public class OrderService {
 }
```

Conceptually:

```
text
 Application startup
        ↓
 Component scanning
        ↓
 Find @Service
        ↓
 Create/register BeanDefinition
        ↓
 BeanFactory knows about OrderService
        ↓
 Dependency resolution
        ↓
 Instantiate OrderService
        ↓
 Apply bean processing
        ↓
 Initialize bean
        ↓
 OrderService becomes managed bean
```

The important thing is:

`@Service` does NOT itself create the object.

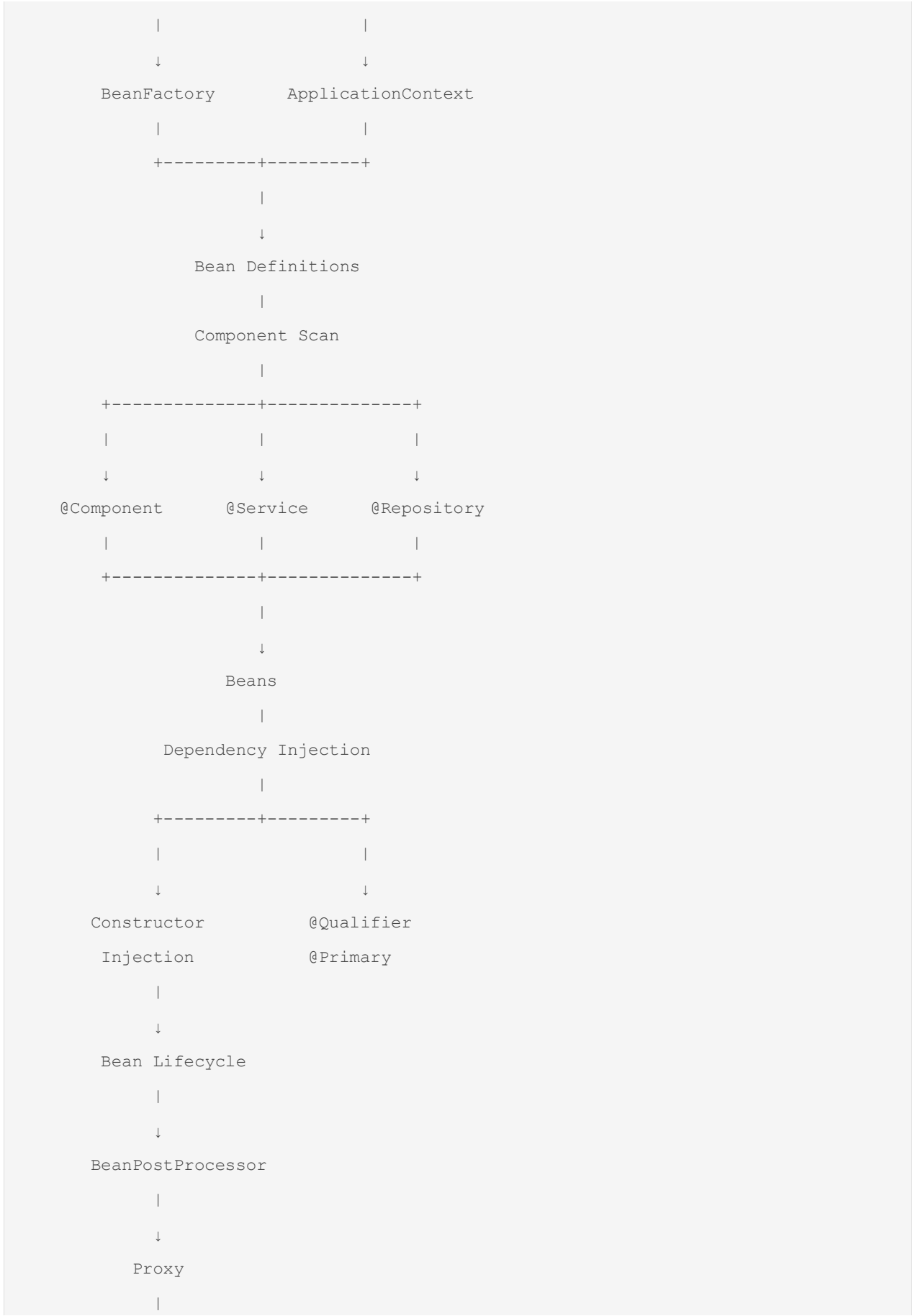
It provides metadata that allows Spring's scanning/configuration machinery to register the class as a bean candidate.

The container then handles creation and management.

□ Part A — What you should now be able to explain

After Q1–Q30, you should be able to draw this from memory:

```
text
                     SPRING
                        |
                        ↓
                IoC CONTAINER
                        |
              +---------+---------+
```


BeanFactory
ApplicationContext
Bean Definitions
Component Scan
@Component
@Service
@Repository
Beans
Dependency Injection
Constructor Injection
@Qualifier
@Primary
Bean Lifecycle
BeanPostProcessor
Proxy

```
        +-----+-----+
        |           |
       AOP      Transactions
```

That is the conceptual foundation for the rest of Spring.

□ The 10 questions from Part A you MUST be able to answer without memorizing

If an interviewer gives you only 10 minutes on Spring Core, these are the ones I'd prioritize:

1. **What is Spring?**
2. **What is IoC?**
3. **What is Dependency Injection?**
4. **BeanFactory vs ApplicationContext?**
5. **What is a Spring Bean?**
6. **Explain Bean Lifecycle.**
7. **Why Constructor Injection?**
8. `@Component` **vs** `@Service` **vs** `@Repository`**?**
9. **What happens when multiple beans implement the same interface?**
10. **What is a Spring Proxy and why is it important?**

And the most important chain to understand is:

```text
 IoC
  ↓
 DI
  ↓
 Spring Container
  ↓
 Bean Definition
  ↓
 Bean Creation
  ↓
 Bean Lifecycle
  ↓
 Proxy
  ↓
 AOP
```

```
  ↓
@Transactional / @Async / @Cacheable / Security
```

Next: Part B — Spring Boot Internals (Q31–Q60)

This is where the interview difficulty increases significantly.

We will go into:

- **Q31 — Spring vs Spring Boot**
- **Q32 — Why was Spring Boot introduced?**
- **Q33 — What exactly does `@SpringBootApplication` do?**
- **Q34 — `@SpringBootApplication` = which three annotations?**
- **Q35 — What is auto-configuration?**
- **Q36 — How does Spring Boot decide what to auto-configure?**
- **Q37 — What happens when you add `spring-boot-starter-data-jpa`?**
- **Q38 — What are Spring Boot starters?**
- **Q39 — How does `@ConditionalOnClass` work?**
- **Q40 — How does `@ConditionalOnMissingBean` work?**
- **Q41 — What is `@Conditional`?**
- **Q42 — How does Boot know which auto-configurations to load?**
- **Q43 — What is `AutoConfiguration.imports`?**
- **Q44 — What happens during `SpringApplication.run()`?**
- **Q45 — How does embedded Tomcat start?**
- **Q46 — What happens before the application is ready?**
- **Q47 — What are profiles?**
- **Q48 — `application.properties` vs `application.yml`**
- **Q49 — `@Value` vs `@ConfigurationProperties`**
- **Q50 — How does externalized configuration work?**
- **Q51 — How do you override Spring Boot auto-configuration?**
- **Q52 — Why does Boot "back off" from auto-configuration?**
- **Q53 — How do you debug auto-configuration?**
- **Q54 — What is Actuator?**
- **Q55 — What are health/readiness/liveness endpoints?**
- **Q56 — What happens when your application starts?**
- **Q57 — Why does Boot start successfully even without an explicit Tomcat configuration?**
- **Q58 — Why does adding one dependency suddenly create infrastructure?**

- **Q59 — How would you debug "works locally but not in production"?**
- **Q60 — Explain Spring Boot startup internally from `main()` to ready state.**

The key concept here is **auto-configuration**. Current Spring Boot documentation explicitly describes Boot as automatically configuring the application based on the JAR dependencies on the classpath, while allowing your own configuration to replace/back off the defaults. Running with `--debug` can expose the conditions report explaining why auto-configurations were applied. (Home)

So **Q37 and Q60 will be especially important for your target interviews** because they force you to explain what Spring Boot is actually doing rather than just reciting annotations. (Home)

you asked: ok continu

message time: 2026-08-23 03:26:09

ok continue as aboved part b.

Part B — Spring Boot Internals

Q31–60

- Spring vs Spring Boot
- `@SpringBootApplication`
- auto-configuration
- starters
- conditional configuration
- `@ConditionalOnClass`
- `@ConditionalOnMissingBean`
- configuration properties
- profiles
- application startup
- embedded server
- Actuator
- custom configuration
- overriding Boot defaults
- debugging auto-configuration

Current 2026 interview material specifically emphasizes dependency injection, IoC, auto-configuration, starters, configuration/profiles, and understanding what happens internally when dependencies are added.

Part C — Spring MVC + REST

Q61–85

- DispatcherServlet
- Controller → Service → Repository
- `@RequestMapping`
- `@GetMapping`
- `@PostMapping`
- `@PathVariable`
- `@RequestParam`
- `@RequestBody`
- DTOs
- validation
- `@Valid`
- global exception handling
- `@ControllerAdvice`
- `ResponseEntity`
- HTTP status codes
- idempotency
- pagination
- API versioning

Part D — Production Spring Boot

Q86–105

- Transactions
- `@Transactional`
- propagation
- isolation
- rollback
- proxy behavior
- self-invocation
- transaction boundaries
- N+1
- connection pools
- caching
- logging
- Actuator
- health checks
- configuration/security mistakes

Part E — Actual Interview Scenarios

Q106–120

These will be the **most valuable**.

For example:

> *"Your API is suddenly taking 5 seconds instead of 200 ms. How do you investigate?"*
>
> *"You added @Transactional, but rollback isn't happening. Why?"*
>
> *"Two beans implement the same interface. What happens?"*
>
> *"Your application starts locally but fails in production. How do you debug it?"*
>
> *"Why does adding spring-boot-starter-data-jpa suddenly create database-related infrastructure?"*
>
> *"Your API returns 500 for invalid input. How would you redesign the exception handling?"*
>
> *"Your Spring Boot service has 10,000 DB queries for one API request. What's happening?"*

These are the questions that turn **memorization into interview performance**.

Recent 2-year interview guidance specifically recommends being able to build a small end-to-end feature—**controller → service → repository → DTO → validation → exception handling**—and explain the design decisions, rather than only knowing annotation definitions.

chatgpt response: Absolutely

Absolutely. We'll continue **exactly from Q31**, keeping the same depth and interview-oriented format.

Module 2 — Spring & Spring Boot

Part B — Spring Boot Internals

Q31–Q60

This part is especially important because a **2-year Spring Boot interviewer often moves from "what annotation?" to "what happens internally?"**

The mental model we want by the end is:

```
text
 Spring Framework
        ↓
```

```
Spring Boot
       ↓
SpringApplication.run()
       ↓
ApplicationContext
       ↓
Component Scanning
       ↓
Auto-Configuration
       ↓
Conditional Evaluation
       ↓
Beans
       ↓
Embedded Server
       ↓
Application Ready
```

Spring Boot's current documentation describes auto-configuration as configuration based on the dependencies present on the classpath, while allowing application-defined configuration to replace or override Boot defaults.

Q31. What is the difference between Spring Framework and Spring Boot?

⭐ Short interview answer

> *Spring Framework provides the core infrastructure such as IoC, Dependency Injection, AOP, transactions, MVC, and data access. Spring Boot is built on top of Spring Framework and simplifies application development by providing auto-configuration, starters, embedded servers, externalized configuration, and production-ready features.*

Don't say:

> *"Spring Boot is a replacement for Spring."*

It isn't.

Think:

```
text
                  Spring Boot
                      ↓
        +-------------+-------------+
```

```
        |             |             |
    Auto-config    Starters    Embedded Server
        |             |             |
        +-------------+-------------+
                      ↓
              Spring Framework
                      ↓
         IoC / DI / AOP / MVC /
         Transactions / Data
```

Q31.1 What problem did Spring Boot solve?

Before Spring Boot, Spring applications could involve significant configuration.

Conceptually:

```
text
 Old Spring application


 Application
    ↓
 Spring configuration
    ↓
 Bean configuration
    ↓
 MVC configuration
    ↓
 Database configuration
    ↓
 Server configuration
    ↓
 Dependency configuration
```

Spring Boot tries to make common application setup **convention-based**.

Instead:

```
text
 Add dependency
       ↓
 Boot detects dependency
       ↓
```

```
Boot applies appropriate configuration
      ↓
Application starts
```

For example:

```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Boot can configure the infrastructure needed for a typical web application.

Q31.2 Does Spring Boot eliminate Spring configuration?

✕No.

It reduces the amount of configuration you need to write.

You can still explicitly configure things.

For example:

```java
@Configuration
public class MyConfig {

    @Bean
    public PaymentService paymentService() {
        return new PaymentService();
    }
}
```

So:

```text
Spring Boot
   =
Spring Framework
   +
Opinionated defaults
   +
```

```
Auto-configuration
    +
Convenience
```

Q32. What is `@SpringBootApplication`?

⭐ Short interview answer

> *@SpringBootApplication is the main Spring Boot application annotation. It combines @SpringBootConfiguration, @EnableAutoConfiguration, and @ComponentScan.*

Conceptually:

```java
@SpringBootApplication
public class Application {
}
```

is approximately:

```java
@SpringBootConfiguration
@EnableAutoConfiguration
@ComponentScan
public class Application {
}
```

Spring Boot's current documentation confirms these three annotations are composed by `@SpringBootApplication`.

Q32.1 What does each part do?

1. `@SpringBootConfiguration`

Indicates that the class can provide Spring Boot configuration.

It is essentially a specialized form of:

```java
@Configuration
```

2. `@EnableAutoConfiguration`

Tells Spring Boot:

> *"Try to automatically configure the application based on the environment and classpath."*

This is the heart of Boot's auto-configuration mechanism.

3. `@ComponentScan`

Tells Spring to scan for components such as:

```text
 @Component
 @Service
 @Repository
 @Controller
 @RestController
```

Q32.2 Interviewer asks:

> *"So @SpringBootApplication creates the application?"*

⭐ Better answer

> *Not by itself. It provides configuration metadata that enables component scanning and auto-configuration. The actual application startup is initiated through SpringApplication.run(...).*

Example:

```java
 public static void main(String[] args) {


     SpringApplication.run(
         Application.class,
         args
     );
 }
```

This distinction is important.

```text
 @SpringBootApplication
         ↓
 configuration metadata


 SpringApplication.run()
```

```
        ↓
application startup
```

Q33. What is Auto-Configuration?

⭐ Short interview answer

> *Spring Boot auto-configuration automatically configures application infrastructure based on the libraries available on the classpath, existing beans, configuration properties, and conditional rules.*

This is one of the **highest-priority Spring Boot interview questions**.

Simple example

Suppose you add:

```
text
spring-boot-starter-web
```

This brings web-related dependencies.

Boot can detect relevant classes and configure things such as:

```
text
Spring MVC
DispatcherServlet
JSON conversion
embedded web server
web infrastructure
```

You don't have to manually configure all of them for a standard application.

Q33.1 How does Boot know what to configure?

This is where the interviewer may go deeper.

Simplified:

```
text
Application starts
       ↓
Boot sees classpath
       ↓
Finds relevant dependencies
```

```
        ↓
Loads auto-configuration candidates
        ↓
Evaluates conditions
        ↓
Condition matches?
       / \
     YES  NO
      ↓    ↓
Configure Skip
      |
      ↓
Create beans
```

For example:

```
text
JPA dependency exists?
        ↓
YES
        ↓
JPA auto-configuration candidate
        ↓
Required classes present?
        ↓
Existing user configuration?
        ↓
No conflicting bean?
        ↓
Configure JPA infrastructure
```

Q34. What are Spring Boot Starters?

⭐ Short interview answer

> *Starters are convenient dependency descriptors that bring together the dependencies commonly required for a particular type of application or feature.*

For example:

```text
spring-boot-starter-web
```

is intended for web applications.

Other examples include:

```text
spring-boot-starter-data-jpa
spring-boot-starter-security
spring-boot-starter-validation
spring-boot-starter-test
```

Q34.1 Why are starters useful?

Without starters, you might manually manage many dependencies:

```text
Spring MVC
Jackson
Validation
Tomcat
Logging
etc.
```

A starter provides a convenient curated dependency set.

Conceptually:

```text
starter-web
     |
     +-- Spring MVC
     +-- JSON support
     +-- embedded server
     +-- related dependencies
```

This also helps keep compatible dependency versions aligned through Spring Boot's dependency management.

Q34.2 Does a starter itself configure the application?

❌Important distinction

A starter primarily provides **dependencies**.

Auto-configuration provides the actual automatic configuration.

Think:

```text
 Starter
    ↓
 puts libraries on classpath
    ↓
 Auto-configuration
    ↓
 detects libraries
    ↓
 configures infrastructure
```

This is a very important interview distinction.

Q35. Explain the relationship between Starters and Auto-Configuration.

⭐ Strong interview answer

> *A starter primarily brings the required libraries onto the application's classpath. Once those libraries are present, Spring Boot's auto-configuration mechanism can detect them and conditionally configure appropriate beans and infrastructure.*

Example:

```text
 spring-boot-starter-data-jpa
           ↓
 JPA/Hibernate dependencies
           ↓
 Classes available on classpath
           ↓
 JPA auto-configuration matches
           ↓
 EntityManagerFactory /
 DataSource-related infrastructure
           ↓
 Application can use Spring Data JPA
```

This distinction is commonly tested.

Q36. What is Conditional Configuration?

⭐ Short interview answer

> *Conditional configuration means Spring only registers a configuration or bean when specified conditions are satisfied. Spring Boot heavily uses conditional annotations to make auto-configuration safe and flexible.*

Example:

```
java
 @Configuration
 @ConditionalOnClass(SomeLibrary.class)
 public class MyAutoConfiguration {
 }
```

Meaning:

```
text
 Is SomeLibrary available?
        |
       YES
        ↓
 Enable configuration


       NO
        ↓
 Skip configuration
```

Q36.1 Why does Spring Boot need conditional configuration?

Because Boot cannot assume every application has the same environment.

Imagine:

```
text
 Application A
     ↓
 JPA present


 Application B
```

```
    ↓
JPA absent


Application C
    ↓
Redis present


Application D
    ↓
Redis absent
```

Boot needs to adapt.

Therefore:

```text
Condition matches
      ↓
configure


Condition doesn't match
      ↓
don't configure
```

This is what makes auto-configuration practical.

Q37. What is `@ConditionalOnClass`?

⭐ Short interview answer

> *@ConditionalOnClass makes a configuration or bean conditional on the presence of specified classes on the application's classpath.*

Example:

```java
@Configuration
@ConditionalOnClass(
    name = "com.example.SomeClient"
)
public class SomeAutoConfiguration {
}
```

Conceptually:

```text
 Is SomeClient.class available?
             |
         +---+---+
         |       |
        YES      NO
         |       |
         ↓       ↓
  enable       skip
```

Q37.1 Why is this useful?

Suppose Boot has configuration for Redis.

You don't want Redis infrastructure automatically configured in an application that doesn't have Redis dependencies.

So Boot can effectively say:

```text
 Redis classes present?
        ↓
 YES → Redis auto-configuration may apply


 NO → skip it
```

Q37.2 Is `@ConditionalOnClass` checking whether a bean exists?

✕No.

It checks whether classes are available on the **classpath**.

Compare:

```text
 @ConditionalOnClass
          ↓
 classpath condition
```

versus:

```text
 @ConditionalOnBean
         ↓
 bean/container condition
```

This distinction is important.

Q38. What is `@ConditionalOnMissingBean`?

⭐ Short interview answer

> *@ConditionalOnMissingBean tells Spring Boot to configure a bean only if a matching bean does not already exist in the application context.*

Example:

```java
 @Bean
 @ConditionalOnMissingBean
 public PaymentService paymentService() {
     return new PaymentService();
 }
```

Conceptually:

```text
 Does PaymentService bean already exist?
              |
        +-----+-----+
        |           |
       YES          NO
        |           |
        ↓           ↓
    Don't create   Create
```

Q38.1 Why is this so important for Boot?

This is how Boot avoids aggressively overriding your configuration.

Suppose Boot provides:

```text
 Default DataSource
```

but you define your own:

```java
@Bean
public DataSource dataSource() {
    ...
}
```

Boot can detect:

```text
User DataSource exists
        ↓
Back off
        ↓
Don't create default
```

This is one of the fundamental ideas behind Boot's **"opinionated defaults, but user configuration wins"** approach.

Q39. What does "Spring Boot backs off" mean?

⭐ Interview answer

> *"Back off" means Spring Boot's auto-configuration condition detects that the application has already provided the relevant configuration or bean, so Boot does not create its default configuration.*

Example:

```text
Boot default
     ↓
Default DataSource
```

You provide:

```java
@Bean
DataSource customDataSource() {
    ...
}
```

Then:

```text
 Existing DataSource found
        ↓
 Boot condition fails
        ↓
 Boot backs off
        ↓
 Your DataSource remains
```

This is one of the most important internal concepts in Spring Boot.

Q40. What is `@Conditional`?

⭐ Short interview answer

> *@Conditional allows a configuration component to be registered only when a specified condition evaluates to true. Spring Boot builds many specialized conditional annotations on top of this mechanism.*

Examples:

```text
 @ConditionalOnClass
 @ConditionalOnMissingBean
 @ConditionalOnBean
 @ConditionalOnProperty
 @ConditionalOnWebApplication
 @ConditionalOnExpression
```

Conceptually:

```text
 @Conditional
     |
     +-- class condition
     +-- bean condition
     +-- property condition
     +-- web application condition
```

Q41. What happens when you add `spring-boot-starter-data-jpa`?

 Very important interview question.

The interviewer may literally ask:

> *"You only added one dependency. Why does Spring Boot suddenly start configuring JPA/Hibernate/database infrastructure?"*

⭐ Strong answer

> *The starter adds JPA-related dependencies to the classpath. During startup, Spring Boot evaluates its JPA-related auto-configuration. If the required classes are available and the relevant conditions are satisfied, Boot configures the JPA infrastructure, using application properties and existing beans where applicable. Auto-configuration also backs off when the application provides conflicting custom configuration.*

Conceptually:

```text
 Add starter
     ↓
 JPA dependencies
     ↓
 Classes available
     ↓
 Auto-configuration candidates
     ↓
 Conditional evaluation
     ↓
 JPA configuration
     ↓
 EntityManagerFactory / transaction infrastructure
     ↓
 Spring Data repositories
```

The exact infrastructure depends on the Boot version, dependencies, and application configuration.

Q41.1 Does adding `spring-boot-starter-data-jpa` automatically connect to any database?

❌No.

This is a common trap.

You generally still need a database driver and appropriate configuration.

For example:

```text
 JPA starter
      ↓
 JPA/Hibernate infrastructure
```

but:

```text
 Database driver
      ↓
 actual database connectivity
```

And configuration such as:

```properties
 spring.datasource.url=...
 spring.datasource.username=...
 spring.datasource.password=...
```

may be needed, depending on your setup.

Q42. What is configuration properties in Spring Boot?

There are several ways to configure applications.

For example:

```properties
 server.port=8081
```

or:

```properties
 payment.timeout=5000
 payment.retry-count=3
```

Spring Boot provides externalized configuration so that application behavior can be changed without hardcoding environment-specific values.

Q42.1 What is `@Value`?

Example:

```java
 @Value("${payment.timeout}")
 private int timeout;
```

If:

```properties
 payment.timeout=5000
```

then:

```text
 timeout = 5000
```

Q42.2 What is `@ConfigurationProperties`?

For grouped configuration:

```properties
 payment.timeout=5000
 payment.retry-count=3
 payment.enabled=true
```

you can model it:

```java
 @ConfigurationProperties(prefix = "payment")
 public class PaymentProperties {

     private int timeout;
     private int retryCount;
     private boolean enabled;

     // getters/setters
 }
```

Then:

```text
 payment.*
    ↓
 PaymentProperties
```

```
    ↓
typed configuration
```

This is generally cleaner for larger groups of related properties.

Q43. `@Value` vs `@ConfigurationProperties`

⭐ Interview answer

`@Value`	`@ConfigurationProperties`
Good for individual values	Good for groups of properties
Simple	Structured
SpEL support	Type-safe binding
Can become scattered	Centralized configuration
Convenient for small values	Better for configuration objects

Example:

```java
@Value("${payment.timeout}")
private int timeout;
```

versus:

```java
@ConfigurationProperties(prefix = "payment")
public class PaymentProperties {
    private int timeout;
    private int retryCount;
}
```

For a production application with many related settings:

> *I'd generally prefer @ConfigurationProperties.*

Q44. What are Spring Profiles?

⭐ Short interview answer

> *Profiles allow different beans and configuration to be activated for different environments or deployment scenarios, such as development, testing, and production.*

Example:

```
text
 application-dev.properties
 application-test.properties
 application-prod.properties
```

Then:

```
text
 dev
  ↓
 development configuration


 test
  ↓
 test configuration


 prod
  ↓
 production configuration
```

Q44.1 How do you activate a profile?

For example:

```
properties
 spring.profiles.active=dev
```

or through environment-specific deployment configuration.

You can also use:

```
java
 @Profile("dev")
 @Bean
 public PaymentService mockPaymentService() {
     return new MockPaymentService();
 }
```

Then the bean is active only when the `dev` profile is active.

Spring's environment/profile system allows bean definitions to be conditionally registered based on active profiles.

Q44.2 Real production example

You might have:

```text
 DEV
  ↓
 localhost DB
 debug logging
 mock external services

 TEST
  ↓
 test DB
 test credentials

 PROD
  ↓
 production DB
 production credentials
 INFO/WARN logging
 real external services
```

Profiles help separate these configurations.

But don't put actual secrets directly into Git just because you use profiles.

That's a production security mistake.

Q45. What is Spring Boot application startup?

⭐ Interview answer

At a high level:

```text
 main()
  ↓
 SpringApplication.run()
```

```
 ↓
Create/configure ApplicationContext
 ↓
Prepare Environment
 ↓
Load configuration
 ↓
Determine application type
 ↓
Register sources
 ↓
Component scanning
 ↓
Auto-configuration
 ↓
Bean creation
 ↓
Embedded server startup
 ↓
Application ready
```

The exact internals are more detailed, but this is the mental model you should know first.

Q45.1 What does `SpringApplication.run()` do?

Example:

```java
public static void main(String[] args) {

    SpringApplication.run(
        Application.class,
        args
    );
}
```

It bootstraps the Spring application.

Conceptually:

```text
 SpringApplication.run()
         ↓
 Prepare environment
         ↓
 Create ApplicationContext
         ↓
 Apply initializers/listeners
         ↓
 Load configuration
         ↓
 Refresh context
         ↓
 Create beans
         ↓
 Start web server if applicable
         ↓
 Publish ready event
```

This is one of the most useful things to understand internally.

Q46. How does Spring Boot know whether it is a web application?

Spring Boot examines the classpath and configuration/application type.

For example, if servlet web infrastructure is present, Boot can create a servlet-based web application context.

Conceptually:

```text
 Classpath
    |
    +-- MVC/Servlet classes?
           |
          YES
           ↓
 Servlet web application
```

Different application types can be supported depending on the dependencies present.

Q47. How does the embedded server work?

Suppose:

```xml
 spring-boot-starter-web
```

is included.

A typical servlet-based Boot application can run with an embedded server such as Tomcat.

Instead of:

```text
 Build WAR
     ↓
 Install external Tomcat
     ↓
 Deploy WAR
```

you commonly have:

```text
 Spring Boot application
        ↓
 embedded Tomcat
        ↓
 java -jar application.jar
```

This is one of Spring Boot's biggest conveniences.

Q47.1 Does Spring Framework itself always provide embedded Tomcat?

✕No.

Embedded server support is a Spring Boot convenience, and the particular server depends on the dependencies/configuration.

For a standard servlet-based application:

```text
 spring-boot-starter-web
         ↓
 web dependencies
```

```
        ↓
embedded servlet container
```

Q48. What happens when the embedded Tomcat starts?

Simplified:

```
text
Spring Boot
    ↓
Creates web application context
    ↓
Creates embedded server
    ↓
Configure server
    ↓
Register Spring MVC infrastructure
    ↓
Start Tomcat
    ↓
Tomcat listens on configured port
```

Then an HTTP request:

```
text
Client
  ↓
Tomcat
  ↓
Servlet infrastructure
  ↓
DispatcherServlet
  ↓
Controller
```

This leads directly into **Part C**, where we'll go deeply into `DispatcherServlet`.

Q49. What is Spring Boot Actuator?

⭐ Short interview answer

> *Spring Boot Actuator provides production-oriented endpoints and monitoring capabilities for observing and managing a Spring Boot application.*

Common examples include endpoints related to:

```
text
 health
 metrics
 info
 beans
 env
 loggers
```

depending on configuration/exposure.

For example:

```
text
 /actuator/health
```

can provide health information.

Q49.1 Why is Actuator important in production?

Imagine:

```
text
 Application running in AWS
        |
        ↓
 Is service healthy?
        |
        ↓
 /actuator/health
```

Monitoring systems can use health information.

Actuator can also expose useful operational information for troubleshooting.

But:

> *Do not expose sensitive actuator endpoints publicly without proper security.*

For example, an endpoint exposing environment/configuration information could reveal sensitive details if poorly secured.

Q50. What is a health check?

⭐ Interview answer

> *A health check is an endpoint or mechanism used to determine whether an application or one of its critical dependencies is functioning sufficiently for the intended purpose.*

Typical:

```
text
 /actuator/health
```

Conceptually:

```
text
 Load Balancer
       |
       ↓
 /actuator/health
       |
    +--+--+
    |     |
   UP    DOWN
```

Q50.1 Health vs readiness vs liveness

This is increasingly important in containerized environments.

Liveness

Question:

> *"Is the application process alive?"*

If liveness fails, the platform may restart the application.

Readiness

Question:

> *"Is this application ready to receive traffic?"*

If readiness fails, the application may remain running but should not receive traffic.

Think:

```text
Liveness
   ↓
Should process stay alive?

Readiness
   ↓
Should process receive traffic?
```

Spring Boot Actuator supports liveness/readiness health groups in environments where those concepts are relevant.

Q51. How do you customize Spring Boot configuration?

Several ways.

`application.properties`

```properties
server.port=8081
```

`application.yml`

```yaml
server:
  port: 8081
```

`@Configuration`

```java
@Configuration
public class AppConfig {
}
```

`@Bean`

```java
@Bean
public PaymentService paymentService() {
    return new PaymentService();
}
```

`@ConfigurationProperties`

```java
@ConfigurationProperties(prefix = "payment")
public class PaymentProperties {
}
```

Custom auto-configuration

For libraries/framework-level development, you can create auto-configuration using Boot's supported mechanisms.

Q52. How do you override Spring Boot's default configuration?

⭐ Interview answer

> *Spring Boot's auto-configuration is designed to back off when application-defined configuration satisfies the relevant conditions. We can also use explicit configuration properties, custom beans, exclusions, or specific configuration mechanisms depending on what we want to change.*

Example:

Boot provides:

```text
default DataSource
```

You provide:

```java
@Bean
public DataSource dataSource() {
    return customDataSource();
}
```

Then:

```text
Custom bean exists
      ↓
@ConditionalOnMissingBean fails
      ↓
Boot backs off
```

Q52.1 Can you disable a particular auto-configuration?

Yes.

For example:

```java
@SpringBootApplication(
    exclude = {
        DataSourceAutoConfiguration.class
    }
)
public class Application {
}
```

Or through configuration properties in appropriate cases.

But don't disable auto-configuration just to make an error disappear.

The interviewer may ask:

> *"Why did you disable it?"*

You should be able to explain the underlying conflict.

Q53. How do you debug Spring Boot auto-configuration?

 Very common advanced interview question.

⭐ Best answer

> *I would first enable the condition evaluation report, typically using --debug, and inspect why an auto-configuration matched or did not match. I'd then inspect the application context, configuration properties, classpath dependencies, and existing beans.*

For example:

```bash
java -jar app.jar --debug
```

Boot can produce the conditions report showing positive and negative matches for auto-configuration.

Q53.1 What are you looking for in the condition report?

Something conceptually like:

```text
 DataSourceAutoConfiguration
    matched:
       required classes found

    did not match:
       embedded database missing
```

or:

```text
 SomeAutoConfiguration
    did not match:
       @ConditionalOnMissingBean
       found existing bean
```

This can tell you:

```text
 Why did Boot configure this?
 Why didn't Boot configure this?
 Which condition failed?
 Which bean already exists?
 Which class is missing?
```

Q54. What is custom configuration?

Suppose Boot's default behavior doesn't fit your application.

You can explicitly configure something:

```java
 @Configuration
 public class PaymentConfig {

     @Bean
     public PaymentClient paymentClient() {

         return new PaymentClient(
             "https://payment-service"
         );
```

```
    }
}
```

Now:

```
text
Spring Boot defaults
        ↓
your custom configuration
        ↓
application-specific behavior
```

Spring Boot is not intended to prevent customization.

The philosophy is closer to:

> *Convention by default, customization when needed.*

Q55. How would you investigate "application works locally but fails in production"?

 This is a real interview scenario.

Don't answer:

> *"Check the logs."*

That's too shallow.

⭐ Strong 2-year answer

I'd investigate systematically:

```
text
1. Application logs
        ↓
2. Stack trace/root exception
        ↓
3. Active Spring profile
        ↓
4. Environment variables
        ↓
5. Configuration properties
        ↓
```

```
6. Database connectivity
        ↓
7. External service connectivity
        ↓
8. Dependency/version differences
        ↓
9. Network/security/firewall
        ↓
10. Resource limits
```

For Spring Boot specifically, I'd check:

```text
Active profiles
Configuration precedence
Missing environment variables
Bean creation failures
Auto-configuration condition report
Database configuration
Actuator health
```

Q56. What is configuration precedence?

Spring Boot supports multiple sources of configuration.

For example:

```text
application.properties
environment variables
command-line arguments
external configuration
```

The exact precedence depends on the source and Boot version.

The key interview concept:

> *A property can come from multiple sources, and higher-priority configuration can override lower-priority values.*

For production, this allows:

```text
 application.properties
         ↓
 defaults

 Environment variables
         ↓
 production override
```

This avoids putting environment-specific values directly into the application artifact.

Q57. Why shouldn't production secrets be hardcoded in `application.properties`?

Suppose:

```properties
 spring.datasource.password=MyPassword123
```

and you commit it to Git.

Now:

```text
 Git repository
        ↓
 credentials exposed
        ↓
 security incident
```

Instead, use appropriate secret-management/environment mechanisms.

For example:

```text
 Environment variables
 Secret manager
 Kubernetes Secrets
 Cloud secret management
```

The exact mechanism depends on infrastructure.

This is a very common production follow-up.

Q58. Why does Spring Boot start so much infrastructure automatically?

This is the **big conceptual question** behind Part B.

⭐ Strong answer

> *Because Spring Boot combines dependency detection, auto-configuration, conditional configuration, and opinionated defaults. When a relevant library is present, Boot finds matching auto-configuration and evaluates conditions. If those conditions match and the application hasn't already provided conflicting configuration, Boot creates the required infrastructure beans.*

Think:

```text
 Dependency
     ↓
 Classpath
     ↓
 Auto-configuration candidate
     ↓
 Conditions
     ↓
 Match?
    / \
  YES  NO
   ↓    ↓
 Beans  Skip
   ↓
 Infrastructure
```

This is the answer you should give instead of:

> *"Spring Boot does it automatically."*

Q59. Explain Spring Boot startup from `main()` to application ready.

□□ **Extremely high-value question.**

Suppose:

```java
 @SpringBootApplication
 public class Application {
```

```
    public static void main(String[] args) {

        SpringApplication.run(
            Application.class,
            args
        );
    }
}
```

The interviewer asks:

> *"What happens internally?"*

⭐ Strong conceptual answer

```
text
main()
  ↓
SpringApplication.run()
  ↓
Create SpringApplication / startup infrastructure
  ↓
Prepare Environment
  ↓
Load configuration
  ↓
Determine application type
  ↓
Create ApplicationContext
  ↓
Register configuration sources
  ↓
Process @SpringBootApplication
  ↓
Component scanning
  ↓
Import auto-configurations
  ↓
Evaluate conditions
```

```
  ↓
Register bean definitions
  ↓
Create singleton beans
  ↓
Dependency injection
  ↓
BeanPostProcessors
  ↓
Initialize beans
  ↓
Start embedded server
  ↓
ApplicationReadyEvent
  ↓
Application ready
```

Don't claim this is the exact low-level call sequence. It's the **correct interview-level lifecycle model**.

Q59.1 What is `ApplicationContext.refresh()`?

This is where you can impress the interviewer.

The context refresh is a major lifecycle operation in Spring's application context.

Conceptually, during refresh Spring performs tasks such as:

```
text
Prepare context
       ↓
Obtain/prepare BeanFactory
       ↓
Process bean definitions
       ↓
Register/process post-processors
       ↓
Initialize infrastructure
       ↓
Instantiate non-lazy singleton beans
       ↓
Finish context initialization
```

```
        ↓
 Publish lifecycle events
```

You don't need to memorize every internal method name for a 2-year interview.

But knowing that **context refresh is central to bean initialization** is valuable.

Q60. Why is understanding Spring Boot internals important for a 2-year developer?

This is the final Part B interview question.

Weak developer:

> *"I know @Autowired, @Service, @RestController and @Transactional."*

Stronger developer:

> *"I understand how the container discovers beans, resolves dependencies, initializes them, applies post-processors/proxies, and how Boot adds auto-configuration on top of the Spring container."*

Product-company-level answer:

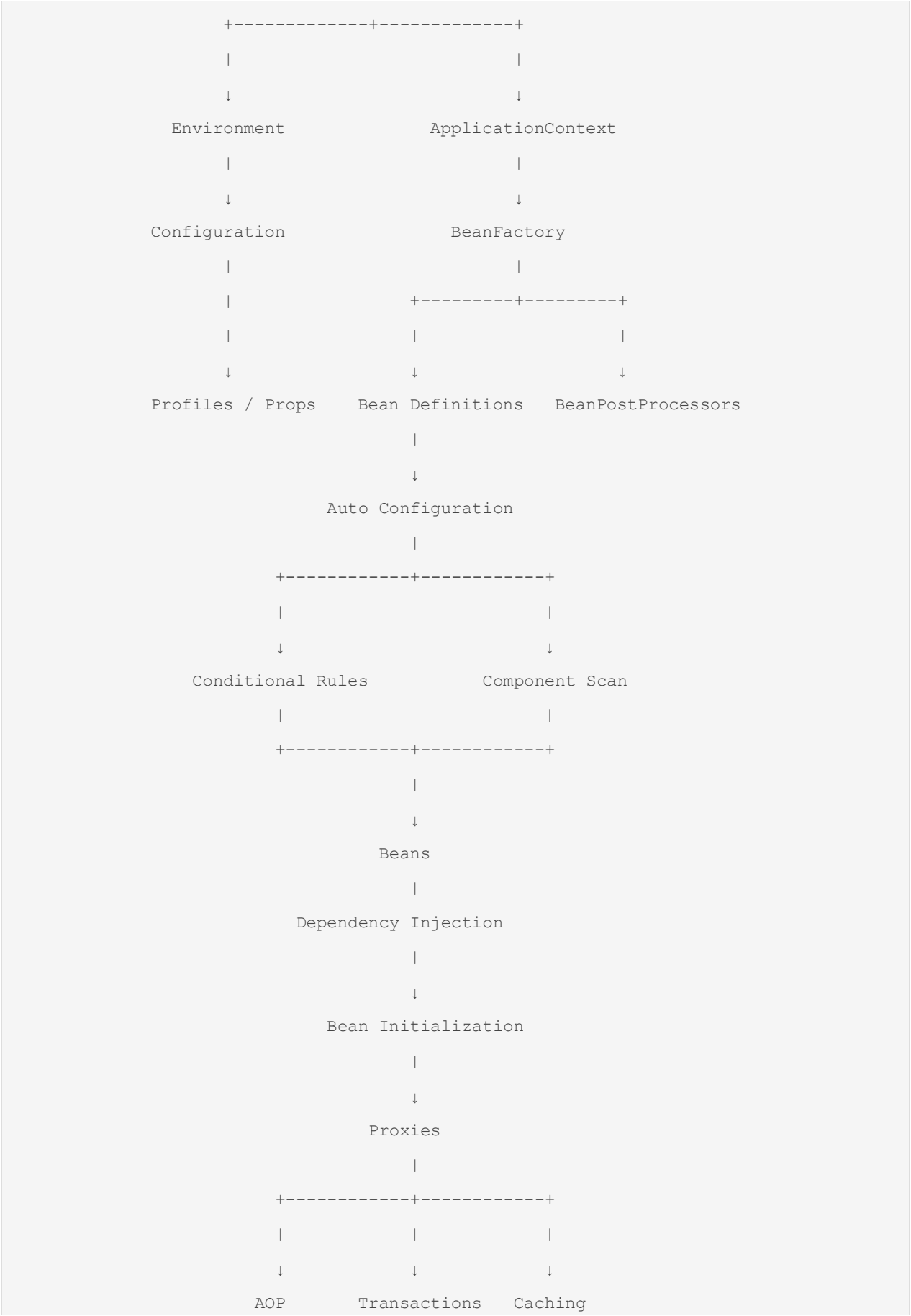
> *At two years of experience, I wouldn't claim to know every Spring internal implementation detail, but I should understand the lifecycle and the reasons behind the framework behavior. If something fails, such as a missing bean, ambiguous bean, unexpected auto-configuration, transaction issue, or startup failure, I should be able to reason from the container, classpath, configuration, conditions, and bean lifecycle rather than trial-and-error annotations.*

That is exactly the mindset interviewers are looking for.

□ Part B — The Most Important Internal Diagram

You should be able to reproduce this on a whiteboard:

```
text
                          Spring Boot Application
                                   |
                                   ↓
                            main() method
                                   |
                                   ↓
                        SpringApplication.run()
                                   |
```


Environment
ApplicationContext
Configuration
BeanFactory
Profiles / Props
Bean Definitions
BeanPostProcessors
Auto Configuration
Conditional Rules
Component Scan
Beans
Dependency Injection
Bean Initialization
Proxies
AOP
Transactions
Caching

```
                    |
                    ↓
            Embedded Server
                    |
                    ↓
            Application Ready
```

□ The 15 Part-B questions I would prioritize

If you have limited preparation time, master these first:

Priority	Question	Why
⭐⭐⭐	Q31 Spring vs Spring Boot	Foundation
⭐⭐⭐	Q32 `@SpringBootApplication`	Extremely common
⭐⭐⭐	Q33 Auto-configuration	Core Boot concept
⭐⭐⭐	Q34 Starters	Common
⭐⭐⭐	Q35 Starters vs Auto-config	Tests understanding
⭐⭐⭐	Q36 Conditional configuration	Internals
⭐⭐⭐	Q37 `@ConditionalOnClass`	Internals
⭐⭐⭐	Q38 `@ConditionalOnMissingBean`	Internals
⭐⭐⭐	Q39 "Back off"	Very important
⭐⭐	Q43 `@Value` vs `@ConfigurationProperties`	Production
⭐⭐⭐	Q44 Profiles	Common
⭐⭐⭐	Q45 Startup	Very important
⭐⭐⭐	Q47 Embedded server	Common
⭐⭐⭐	Q53 Debug auto-config	Strong differentiator
⭐⭐⭐	Q59 Startup from `main()`	Product-company depth

□ One interview scenario combining Part A + Part B

Interviewer:

> *"I created a Spring Boot application. I added spring-boot-starter-data-jpa. I didn't create a DataSource bean manually. Yet Spring starts creating database-related infrastructure. Explain what is happening."*

⭐ Excellent 2-year answer

> *"The starter brings the JPA-related dependencies onto the classpath. During application startup, SpringApplication.run() creates and refreshes the application context. Spring Boot imports its auto-configuration candidates and evaluates their conditions. Because the relevant JPA classes are present, the JPA-related auto-configuration can match. Boot then configures the required infrastructure based on the application's configuration and available beans. If I define my own compatible infrastructure, conditions such as @ConditionalOnMissingBean can cause Boot's default configuration to back off. So the starter provides dependencies, while auto-configuration uses those dependencies to conditionally create infrastructure."*

That answer demonstrates:

```text
 Starter
    ↓
 Classpath
    ↓
 SpringApplication
    ↓
 ApplicationContext
    ↓
 Auto-configuration
    ↓
 Conditions
    ↓
 Beans
    ↓
 Back-off
```

That's the level we want for the rest of this interview bible.

Next: Part C — Spring MVC + REST Q61–Q85

This is where we'll take the same internal approach and follow an actual HTTP request:

```text
 Client
   |
```

```
  | GET /orders/123
  ↓
Embedded Tomcat
  ↓
DispatcherServlet
  ↓
HandlerMapping
  ↓
Controller
  ↓
Service
  ↓
Repository
  ↓
Database
  ↓
Repository
  ↓
Service
  ↓
Controller
  ↓
HttpMessageConverter
  ↓
JSON
  ↓
HTTP Response
  ↓
Client
```

And we'll cover **Q61–Q85** including `DispatcherServlet`, request mapping, DTOs, validation, `@Valid`, exception handling, `@ControllerAdvice`, `ResponseEntity`, HTTP status codes, idempotency, pagination, and API versioning—while also building the **complete controller → service → repository → DTO → validation → exception-handling flow** that a 2-year product-company interviewer can ask you to implement and explain.

you asked: ok continu

message time: 2026-08-23 03:29:46

ok continue as aboved part c.

Part C — Spring MVC + REST

Q61–85

- DispatcherServlet
- Controller → Service → Repository
- `@RequestMapping`
- `@GetMapping`
- `@PostMapping`
- `@PathVariable`
- `@RequestParam`
- `@RequestBody`
- DTOs
- validation
- `@Valid`
- global exception handling
- `@ControllerAdvice`
- `ResponseEntity`
- HTTP status codes
- idempotency
- pagination
- API versioning

Part D — Production Spring Boot

Q86–105

- Transactions
- `@Transactional`
- propagation
- isolation
- rollback
- proxy behavior
- self-invocation
- transaction boundaries
- N+1
- connection pools
- caching

- logging
- Actuator
- health checks
- configuration/security mistakes

Part E — Actual Interview Scenarios

Q106–120

These will be the **most valuable**.

For example:

> *"Your API is suddenly taking 5 seconds instead of 200 ms. How do you investigate?"*
>
> *"You added @Transactional, but rollback isn't happening. Why?"*
>
> *"Two beans implement the same interface. What happens?"*
>
> *"Your application starts locally but fails in production. How do you debug it?"*
>
> *"Why does adding spring-boot-starter-data-jpa suddenly create database-related infrastructure?"*
>
> *"Your API returns 500 for invalid input. How would you redesign the exception handling?"*
>
> *"Your Spring Boot service has 10,000 DB queries for one API request. What's happening?"*

These are the questions that turn **memorization into interview performance**.

Recent 2-year interview guidance specifically recommends being able to build a small end-to-end feature—**controller → service → repository → DTO → validation → exception handling**—and explain the design decisions, rather than only knowing annotation definitions.

chatgpt response: # Module 2

Module 2 — Spring & Spring Boot

Part C — Spring MVC + REST

Q61–Q85

Now we move from **"how Spring Boot starts"** to **"what happens when an HTTP request enters your application."**

For a 2-year Java/Spring Boot interview, don't memorize annotations independently. You should understand this complete flow:

```text
Client
   |
   | HTTP Request
   ↓
Embedded Tomcat
   ↓
DispatcherServlet
   ↓
HandlerMapping
   ↓
Controller
   ↓
Service
   ↓
Repository
   ↓
Database
   ↓
Repository
   ↓
Service
   ↓
Controller
   ↓
HttpMessageConverter
   ↓
JSON Response
   ↓
Client
```

That mental model will help you answer most of Q61–Q85.

Q61. What is `DispatcherServlet`?

 Extremely important Spring MVC question.

⭐ Short interview answer

> *DispatcherServlet is the front controller of Spring MVC. It receives incoming HTTP requests and delegates them to the appropriate controller/handler through Spring MVC's request-processing infrastructure.*

Think:

```text
                  Client
                    |
                    | HTTP Request
                    ↓
             DispatcherServlet
                    |
           +--------+--------+
           |        |        |
           ↓        ↓        ↓
     Controller  Handler   Exception
                 Mapping    Handling
```

It is not your business logic.

It acts as the **central entry point for Spring MVC request processing**.

Q61.1 Why is it called "Front Controller"?

Because instead of every controller independently handling infrastructure concerns, requests first pass through a central component.

```text
 Without Front Controller


 Request
   ↓
 Controller A


 Request
   ↓
 Controller B


 Request
   ↓
 Controller C
```

With Spring MVC:

```
text
                    Request
                       ↓
                DispatcherServlet
                       ↓
            +----------+----------+
            ↓          ↓          ↓
       Controller A Controller B Controller C
```

The central servlet can coordinate:

- handler mapping
- request processing
- argument resolution
- message conversion
- exception handling
- response processing

Q61.2 Explain the request flow internally.

Suppose the client calls:

```
http
 GET /users/101
```

Conceptually:

```
text
 1. Client sends request
           ↓
 2. Tomcat receives request
           ↓
 3. DispatcherServlet receives it
           ↓
 4. HandlerMapping finds matching controller
           ↓
 5. HandlerAdapter invokes controller
           ↓
 6. Controller calls service
```

```
          ↓
7. Service calls repository
          ↓
8. Repository accesses database
          ↓
9. Service returns result
          ↓
10. Controller returns object
          ↓
11. HttpMessageConverter serializes object
          ↓
12. HTTP response sent
```

This sequence is **very important**.

Q62. What is the Controller → Service → Repository architecture?

⭐ Short interview answer

> *Controller handles HTTP concerns, Service contains business logic, and Repository handles persistence/data access. Separating these responsibilities makes the application easier to maintain, test, and evolve.*

Typical structure:

```text
Controller
    ↓
Service
    ↓
Repository
    ↓
Database
```

Example:

```java
@RestController
@RequestMapping("/users")
public class UserController {

    private final UserService userService;
```

```
    public UserController(UserService userService) {
        this.userService = userService;
    }

    @GetMapping("/{id}")
    public UserResponse getUser(@PathVariable Long id) {
        return userService.getUser(id);
    }
}
```

Service:

```java
@Service
public class UserService {

    private final UserRepository repository;

    public UserService(UserRepository repository) {
        this.repository = repository;
    }

    public UserResponse getUser(Long id) {

        User user = repository.findById(id)
                .orElseThrow(() ->
                    new UserNotFoundException(id));

        return new UserResponse(
                user.getId(),
                user.getName()
        );
    }
}
```

Repository:

```java
@Repository
```

```
public interface UserRepository
        extends JpaRepository<User, Long> {
}
```

Q62.1 Why shouldn't we put business logic inside Controller?

Bad:

```java
@GetMapping("/{id}")
public UserResponse getUser(@PathVariable Long id) {

    User user = repository.findById(id).orElseThrow();

    if (user.getStatus() == ACTIVE) {
        // lots of business logic
    }

    // more logic...
}
```

The controller becomes responsible for:

```text
HTTP
+
validation
+
business logic
+
database access
+
response formatting
```

That's difficult to maintain.

Better:

```text
Controller
   ↓
```

```
HTTP-related responsibility


Service
   ↓
Business responsibility


Repository
   ↓
Persistence responsibility
```

Q62.2 Is Controller → Service → Repository a mandatory Spring rule?

✕No.

Spring doesn't require exactly three layers.

It's an architectural pattern commonly used in enterprise applications.

For a simple application, you might have fewer layers.

For a complex application, you might have:

```
text
 Controller
  ↓
 Application Service
  ↓
 Domain Service
  ↓
 Repository
  ↓
 Database
```

The important principle is **separation of responsibilities**, not blindly creating classes.

Q63. What is `@RequestMapping`?

⭐ Short interview answer

> *@RequestMapping maps HTTP requests to a controller or controller method based on URL path, HTTP method, parameters, headers, and other request conditions.*

Example:

```java
@RestController
@RequestMapping("/users")
public class UserController {
}
```

Now:

```text
/users
```

is the base path.

Method:

```java
@GetMapping("/{id}")
public UserResponse getUser(
        @PathVariable Long id) {
    ...
}
```

produces:

```http
GET /users/101
```

Q63.1 Can `@RequestMapping` be used at class and method level?

Yes.

```java
@RequestMapping("/users")
public class UserController {

    @RequestMapping(
        value = "/{id}",
        method = RequestMethod.GET
    )
    public User getUser(@PathVariable Long id) {
        ...
```

```
    }
}
```

Equivalent modern style:

```java
@GetMapping("/{id}")
```

Q63.2 What can `@RequestMapping` match?

It can be used with things such as:

```text
path
HTTP method
params
headers
consumes
produces
```

Example:

```java
@RequestMapping(
    value = "/users",
    method = RequestMethod.GET,
    produces = "application/json"
)
```

Q64. What is `@GetMapping`?

⭐ Short answer

> *@GetMapping is a composed annotation used to map HTTP GET requests to a controller method.*

Example:

```java
@GetMapping("/users/{id}")
public UserResponse getUser(
        @PathVariable Long id) {
```

```
    return userService.getUser(id);
}
```

Request:

```
http
GET /users/101
```

Q64.1 Why use `@GetMapping` instead of `@RequestMapping(method = GET)`?

Readability.

Instead of:

```
java
@RequestMapping(
    value = "/users",
    method = RequestMethod.GET
)
```

you write:

```
java
@GetMapping("/users")
```

The intention is immediately obvious.

Similar composed annotations include:

```
text
@GetMapping
@PostMapping
@PutMapping
@PatchMapping
@DeleteMapping
```

Q65. What is `@PostMapping`?

⭐ Short answer

> *@PostMapping maps HTTP POST requests to a controller method, commonly used for creating resources or triggering operations that aren't safely represented by GET.*

Example:

```java
 @PostMapping("/users")
 public ResponseEntity<UserResponse> createUser(
         @RequestBody CreateUserRequest request) {

     UserResponse response =
             userService.createUser(request);

     return ResponseEntity
             .status(HttpStatus.CREATED)
             .body(response);
 }
```

Request:

```http
 POST /users
 Content-Type: application/json
```

Body:

```json
 {
   "name": "Vaibhav",
   "email": "vaibhav@example.com"
 }
```

Q65.1 GET vs POST

GET	**POST**
Retrieve data	Usually create/process data
Parameters often in URL	Data commonly in request body
Safe when correctly designed	Not generally safe
Cacheable in appropriate situations	Usually not cached by default
Should not change server state	Can change server state

Important:

> *Don't say "GET never changes data" as an absolute technical law. The correct design expectation is that GET should be safe and not intentionally perform state-changing operations.*

Q66. What is `@PathVariable`?

⭐ Short answer

> *@PathVariable extracts a value from a URI path and binds it to a controller method parameter.*

Example:

```java
 @GetMapping("/users/{id}")
 public UserResponse getUser(
         @PathVariable Long id) {

     return userService.getUser(id);
 }
```

Request:

```http
 GET /users/101
```

Then:

```text
 id = 101
```

Q66.1 Multiple path variables

```java
 @GetMapping("/users/{userId}/orders/{orderId}")
 public OrderResponse getOrder(
         @PathVariable Long userId,
         @PathVariable Long orderId) {

     return service.getOrder(userId, orderId);
 }
```

Request:

```http
GET /users/10/orders/500
```

Result:

```text
userId = 10
orderId = 500
```

Q66.2 `@PathVariable` vs `@RequestParam`

This is a **very common follow-up**.

Path variable

```http
GET /users/101
```

```java
@PathVariable Long id
```

Usually identifies a specific resource.

Request parameter

```http
GET /users?page=2&size=20
```

```java
@RequestParam int page
@RequestParam int size
```

Usually controls filtering, pagination, sorting, searching, etc.

Think:

```text
/users/101
       ↑
resource identity

/users?page=2
       ↑
query/filter/control
```

Q67. What is `@RequestParam`?

⭐ Short answer

> *@RequestParam binds query-string parameters or form parameters to controller method arguments.*

Example:

```
java
 @GetMapping("/users")
 public List<UserResponse> getUsers(
         @RequestParam int page,
         @RequestParam int size) {

     return userService.getUsers(page, size);
 }
```

Request:

```
http
 GET /users?page=0&size=20
```

Q67.1 Optional request parameter

```
java
 @GetMapping("/users")
 public List<UserResponse> searchUsers(
         @RequestParam(required = false)
         String name) {

     ...
 }
```

Then both can work:

```
http
 GET /users
```

and:

```
http
 GET /users?name=John
```

You can also provide defaults:

```java
@RequestParam(
    defaultValue = "20"
) int size
```

Q68. What is `@RequestBody`?

⭐ Short answer

> *@RequestBody tells Spring MVC to deserialize the HTTP request body into a Java object using an appropriate HttpMessageConverter.*

Request:

```json
{
  "name": "John",
  "email": "john@example.com"
}
```

Controller:

```java
@PostMapping("/users")
public UserResponse createUser(
        @RequestBody CreateUserRequest request) {

    return userService.createUser(request);
}
```

Conceptually:

```text
JSON
 ↓
HttpMessageConverter
 ↓
CreateUserRequest
```

Q68.1 What does Jackson have to do with this?

For JSON requests/responses, Spring Boot commonly uses Jackson through HTTP message conversion infrastructure.

Conceptually:

```text
 Incoming JSON
       ↓
 Jackson
       ↓
 Java object
```

Response:

```text
 Java object
       ↓
 Jackson
       ↓
 JSON
```

So:

```text
 Request:
 JSON → Java


 Response:
 Java → JSON
```

Q69. What are DTOs?

 Very important for product-company interviews.

⭐ Short interview answer

> *DTO stands for Data Transfer Object. A DTO represents the data transferred across an application boundary, such as an HTTP API, and helps decouple the external API contract from internal domain/entity models.*

Example:

```java
 public class CreateUserRequest {
```

```
    private String name;
    private String email;

    // getters/setters
}
```

Response:

```java
public class UserResponse {

    private Long id;
    private String name;
    private String email;

    // getters/setters
}
```

Q69.1 Why shouldn't we directly expose JPA Entity?

Suppose:

```java
@Entity
public class User {

    private Long id;
    private String name;
    private String passwordHash;
}
```

If you return:

```java
return user;
```

you risk exposing internal fields.

Potentially:

```json
{
  "id": 1,
  "name": "John",
  "passwordHash": "..."
}
```

That's dangerous.

Instead:

```text
Entity
  ↓
Mapper
  ↓
DTO
  ↓
API response
```

Example:

```java
return new UserResponse(
    user.getId(),
    user.getName()
);
```

Q69.2 DTO benefits

DTOs provide:

```text
API contract isolation
Security
Versioning flexibility
Validation separation
Reduced coupling
Controlled response fields
```

For example:

```text
 Database Entity
        |
        | mapping
        ↓
 UserResponse DTO
        |
        ↓
 JSON API
```

If the database entity changes, your API doesn't necessarily have to change.

Q70. What is validation in Spring Boot?

⭐ Short answer

> *Validation checks whether incoming data satisfies defined constraints before business processing occurs. Spring Boot commonly integrates Jakarta Bean Validation with Spring MVC.*

Example DTO:

```java
 public class CreateUserRequest {

     @NotBlank
     private String name;

     @Email
     @NotBlank
     private String email;

     @Size(min = 8)
     private String password;
 }
```

Now invalid input can be rejected before reaching business logic.

Q70.1 Common validation annotations

You should know:

```text
 @NotNull
 @NotBlank
 @NotEmpty
 @Size
 @Min
 @Max
 @Positive
 @PositiveOrZero
 @Email
 @Pattern
 @Past
 @Future
```

Example:

```java
 @NotBlank
 private String name;

 @Email
 private String email;

 @Min(18)
 private int age;
```

Q71. What is `@Valid`?

 Common interview question.

⭐ Short answer

> *@Valid triggers Bean Validation for the object being bound, such as a request DTO. If validation fails, Spring MVC can reject the request before the controller's normal processing continues.*

Example:

```java
 @PostMapping("/users")
 public ResponseEntity<UserResponse> createUser(
         @Valid
```

```
        @RequestBody
        CreateUserRequest request) {


    ...
}
```

Flow:

```
text
HTTP JSON
   ↓
Deserialize DTO
   ↓
@Valid
   ↓
Validation
   |
 +--+--+
 |     |
Valid Invalid
 |     |
 ↓     ↓
Service Exception handling
```

Q71.1 `@Valid` vs `@Validated`

This is a useful follow-up.

`@Valid` is standard Bean Validation cascading validation.

`@Validated` is a Spring-specific annotation that also supports validation groups and is often used for method-level validation.

For example:

```
java
@Validated
@Service
public class UserService {
}
```

You don't need to overcomplicate this in a basic interview.

A good answer:

> *"@Valid is the standard Bean Validation trigger, while @Validated is Spring's variant that supports features such as validation groups and is commonly used for method-level validation."*

Q72. What happens when validation fails?

Suppose:

```
java
 @PostMapping("/users")
 public UserResponse create(
         @Valid @RequestBody CreateUserRequest request) {
     ...
 }
```

and request is:

```
json
 {
   "name": "",
   "email": "wrong"
 }
```

Validation detects:

```
text
 name → @NotBlank FAILED
 email → @Email FAILED
```

Spring MVC raises an appropriate validation-related exception.

Instead of letting the application return an ugly generic error, we should provide a structured API response.

That's where global exception handling comes in.

Q73. What is global exception handling?

⭐ Short interview answer

> *Global exception handling centralizes the translation of application exceptions into consistent HTTP responses instead of handling exceptions individually in every controller.*

Without centralized handling:

```
java
 try {
    ...
 }
 catch (...) {
    ...
 }
```

repeated in every controller.

Bad:

```
text
 Controller A → exception handling
 Controller B → exception handling
 Controller C → exception handling
 Controller D → exception handling
```

Better:

```
text
 Controller A ─┐
 Controller B ─┤
 Controller C ─┼──→ Global Exception Handler
 Controller D ─┘
```

Q74. What is `@ControllerAdvice`?

 Very important.

⭐ Short interview answer

> *@ControllerAdvice allows us to define cross-cutting controller behavior, commonly including centralized exception handling across controllers.*

Example:

```
java
 @RestControllerAdvice
 public class GlobalExceptionHandler {


     @ExceptionHandler(UserNotFoundException.class)
```

```
    public ResponseEntity<ErrorResponse> handleUserNotFound(
            UserNotFoundException ex) {

        ErrorResponse error =
                new ErrorResponse(
                    "USER_NOT_FOUND",
                    ex.getMessage()
                );

        return ResponseEntity
                .status(HttpStatus.NOT_FOUND)
                .body(error);
    }
}
```

`@RestControllerAdvice` is particularly convenient for REST APIs because response bodies are written directly.

Q74.1 What is `@ExceptionHandler`?

It tells Spring which exceptions a method should handle.

```java
@ExceptionHandler(UserNotFoundException.class)
public ResponseEntity<ErrorResponse> handle(
        UserNotFoundException ex) {

    ...
}
```

Flow:

```text
Controller
    ↓
Exception
    ↓
@ControllerAdvice
    ↓
@ExceptionHandler
```

```
    ↓
ErrorResponse
    ↓
HTTP response
```

Q74.2 What should a production error response look like?

Instead of:

```json
{
  "error": "something went wrong"
}
```

you might define:

```json
{
  "timestamp": "2026-08-23T10:30:00Z",
  "status": 404,
  "code": "USER_NOT_FOUND",
  "message": "User not found",
  "path": "/api/users/101",
  "traceId": "abc-123"
}
```

You don't necessarily need every field, but the important principle is:

> *Make API errors predictable and machine-readable.*

Also, don't expose internal stack traces or database errors to clients.

Q75. What is `ResponseEntity`?

⭐ Short interview answer

> *ResponseEntity represents the complete HTTP response, allowing the controller to control the response body, HTTP status, and headers.*

Example:

```java
return ResponseEntity
```

```
        .status(HttpStatus.CREATED)
        .body(userResponse);
```

You can control:

```
text
 Status
 Headers
 Body
```

Q75.1 Why not simply return the DTO?

You can.

Example:

```
java
 @GetMapping("/{id}")
 public UserResponse getUser(...) {
     return service.getUser(id);
 }
```

Spring will typically produce:

```
text
 200 OK
 +
 JSON body
```

If you need explicit control:

```
java
 ResponseEntity<UserResponse>
```

becomes useful.

For example:

```
java
 return ResponseEntity
         .status(HttpStatus.CREATED)
         .header("X-Request-Id", requestId)
         .body(response);
```

Q76. What HTTP status codes should a 2-year Spring developer know?

 Must know.

Success

```
text
 200 OK
 201 Created
 202 Accepted
 204 No Content
```

Client errors

```
text
 400 Bad Request
 401 Unauthorized
 403 Forbidden
 404 Not Found
 409 Conflict
 422 Unprocessable Content
```

Server errors

```
text
 500 Internal Server Error
 502 Bad Gateway
 503 Service Unavailable
 504 Gateway Timeout
```

Q76.1 401 vs 403

 Very common trap.

401 Unauthorized

Usually means:

> *The request lacks valid authentication credentials.*

Example:

```
text
 No valid JWT
```

```
Expired token
Invalid credentials
```

403 Forbidden

Means:

> *The server understood the authenticated identity, but that identity is not allowed to perform the operation.*

Example:

```
text
User authenticated
       ↓
Role = USER
       ↓
Trying admin operation
       ↓
403 Forbidden
```

Q76.2 400 vs 404 vs 409

400

Request itself is invalid.

```
json
{
  "age": "abc"
}
```

404

Resource doesn't exist.

```
http
GET /users/999999
```

if that user doesn't exist.

409

Request conflicts with current state.

Example:

```http
POST /users
```

with an email that must be unique and already exists.

Q77. What is idempotency?

 Very important REST interview concept.

⭐ Short interview answer

> *An operation is idempotent if making the same request multiple times has the same intended server-side effect as making it once.*

Mathematically:

```text
f(f(x)) = f(x)
```

For HTTP semantics, commonly:

```text
GET     → idempotent
PUT     → idempotent
DELETE  → idempotent
POST    → generally not idempotent
```

Q77.1 Example of idempotent PUT

Suppose:

```http
PUT /users/101
```

Body:

```json
{
  "name": "John"
}
```

Sending it once:

```text
User 101 → John
```

Sending it five times:

```text
User 101 → John
```

The intended final state is the same.

Q77.2 Why is POST usually not idempotent?

Suppose:

```http
POST /orders
```

Each request may create a new order:

```text
Request 1 → Order #101
Request 2 → Order #102
Request 3 → Order #103
```

So repeating it can create additional effects.

Q77.3 Real production problem: payment API

Imagine:

```text
Client
  |
  | POST /payments
  ↓
Server
  |
  ↓
Payment succeeds
  |
  X
Network timeout
```

Client thinks:

> *"Payment failed."*

It retries:

```
text
 POST /payments
```

Now payment could happen twice.

A common solution is an **idempotency key**:

```
http
 POST /payments
 Idempotency-Key: abc123
```

Server stores the result associated with that key.

```
text
 Request abc123
      ↓
 Already processed?
     / \
   YES  NO
    ↓    ↓
 Return Process
 existing
 result
```

This is an excellent product-company interview example.

Q78. What is pagination?

⭐ Short interview answer

> *Pagination divides a large result set into smaller responses so the API doesn't return an unnecessarily large amount of data in a single request.*

Instead of:

```
http
 GET /users
```

returning 10 million records:

```http
GET /users?page=0&size=20
```

Response:

```json
{
  "content": [...],
  "page": 0,
  "size": 20,
  "totalElements": 100000,
  "totalPages": 5000
}
```

Q78.1 Why is pagination important?

Without pagination:

```text
Database
   ↓
1,000,000 rows
   ↓
Java heap
   ↓
JSON serialization
   ↓
Huge HTTP response
```

Possible consequences:

```text
High memory usage
Slow response
Network overhead
Database load
Timeouts
```

Pagination limits the amount of data processed per request.

Q78.2 Offset pagination

Example:

```http
GET /users?page=5&size=20
```

Conceptually:

```text
offset = page × size
       = 5 × 20
       = 100
```

So:

```text
skip 100
take 20
```

Easy to implement, but for very large datasets, deep offsets can become inefficient depending on the database/query.

Q78.3 Cursor/keyset pagination

Instead of:

```http
?page=50000
```

you might use:

```http
?limit=20&afterId=10500
```

Conceptually:

```text
last record
     ↓
cursor
     ↓
next query starts after cursor
```

This can perform better for large, ordered datasets.

Q79. How would you design a production pagination API?

A reasonable response:

```
json
 {
   "content": [
     {
       "id": 101,
       "name": "John"
     }
   ],
   "page": 0,
   "size": 20,
   "totalElements": 145,
   "totalPages": 8,
   "hasNext": true
 }
```

Request:

```
http
 GET /api/users?page=0&size=20&sort=name,asc
```

Important production considerations:

```
text
 Maximum page size
 Default page size
 Stable sorting
 Indexes
 Large-offset performance
 Cursor pagination for large datasets
```

Don't allow:

```
http
 ?page=0&size=100000000
```

without protection.

Q80. What is API versioning?

⭐ Short interview answer

> *API versioning allows us to evolve an API contract while maintaining compatibility for existing clients.*

Example:

```
text
 /api/v1/users
 /api/v2/users
```

Q80.1 Why do we need API versioning?

Suppose V1 returns:

```
json
 {
   "name": "John",
   "age": 30
 }
```

Now V2 needs:

```
json
 {
   "firstName": "John",
   "dateOfBirth": "1996-01-01"
 }
```

Changing V1 immediately could break existing clients.

Instead:

```
text
 Existing mobile app
        ↓
 /api/v1/users


 New application
        ↓
 /api/v2/users
```

Both can coexist during migration.

Q81. What are common API versioning strategies?

1. URI versioning

```text
/api/v1/users
/api/v2/users
```

Simple and widely understood.

2. Query parameter

```text
/api/users?version=1
```

3. Header

```http
Accept: application/vnd.company.user-v2+json
```

4. Media-type/content negotiation

Version encoded through media type.

Interview answer

> *"URI versioning is simple and easy for clients and teams to understand, while header/media-type versioning keeps the URI stable. The important part is choosing a consistent strategy and maintaining backward compatibility."*

Q82. What is the difference between `@Controller` and `@RestController`?

 Very common.

`@Controller`

Used for Spring MVC controllers, commonly associated with view rendering.

```java
@Controller
public class UserController {
}
```

`@RestController`

Conceptually combines:

```java
@Controller
@ResponseBody
```

Example:

```java
@RestController
public class UserController {
}
```

The return value is generally written directly to the HTTP response body.

For REST APIs:

```java
@RestController
```

is usually what you use.

Q83. What is `@ResponseBody`?

⭐ Short answer

> *@ResponseBody tells Spring MVC to write the method's return value directly to the HTTP response body rather than resolving it as a view.*

Example:

```java
@Controller
public class UserController {

    @GetMapping("/users")
    @ResponseBody
    public UserResponse getUser() {
        ...
    }
}
```

`@RestController` effectively gives you this behavior for controller methods.

Q84. What are `HttpMessageConverter`s?

□ This is a good **internal-working follow-up**.

⭐ Short interview answer

> *HttpMessageConverters convert HTTP request bodies into Java objects and Java return values into HTTP response bodies based on content type and supported formats.*

Think:

```
text
 Incoming:


 JSON
  ↓
 HttpMessageConverter
  ↓
 Java DTO
```

Outgoing:

```
text
 Java DTO
  ↓
 HttpMessageConverter
  ↓
 JSON
```

For JSON APIs, Jackson is commonly involved.

Q84.1 What happens when `@RequestBody` is used?

Suppose:

```
java
 @PostMapping("/users")
 public UserResponse create(
         @RequestBody CreateUserRequest request) {
     ...
 }
```

Request:

```
http
 Content-Type: application/json
```

Body:

```
json
 {
   "name": "John"
 }
```

Conceptually:

```
text
 HTTP Request
       ↓
 DispatcherServlet
       ↓
 HandlerAdapter
       ↓
 @RequestBody argument resolution
       ↓
 HttpMessageConverter
       ↓
 Jackson
       ↓
 CreateUserRequest
       ↓
 Controller method
```

This is the level of detail that can distinguish you from someone who only knows annotation definitions.

Q85. Design a complete Spring Boot REST API flow.

□□□ **Most important question in Part C.**

Interviewer:

> *"Design a User creation API. Explain the complete request flow from client to database and back."*

Let's build it properly.

Step 1 — Request

Client sends:

```http
POST /api/v1/users
Content-Type: application/json

```

```json
{
  "name": "John",
  "email": "john@example.com",
  "password": "secret123"
}
```

Step 2 — Controller

```java
@RestController
@RequestMapping("/api/v1/users")
public class UserController {

    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

    @PostMapping
    public ResponseEntity<UserResponse> createUser(
            @Valid @RequestBody CreateUserRequest request) {

        UserResponse response =
                userService.createUser(request);

        return ResponseEntity
                .status(HttpStatus.CREATED)
                .body(response);
    }
}
```

Step 3 — DTO

```java
public class CreateUserRequest {

    @NotBlank
    private String name;

    @Email
    @NotBlank
    private String email;

    @Size(min = 8)
    private String password;

    // getters/setters
}
```

Important:

```text
JSON
 ↓
DTO
 ↓
Validation
```

Step 4 — Service

```java
@Service
public class UserService {

    private final UserRepository repository;

    public UserService(UserRepository repository) {
        this.repository = repository;
    }

    public UserResponse createUser(
            CreateUserRequest request) {
```

```
        if (repository.existsByEmail(request.getEmail())) {
            throw new UserAlreadyExistsException(
                    request.getEmail());
        }

        User user = new User();

        user.setName(request.getName());
        user.setEmail(request.getEmail());
        user.setPasswordHash(
                hashPassword(request.getPassword()));

        User saved = repository.save(user);

        return new UserResponse(
                saved.getId(),
                saved.getName(),
                saved.getEmail()
        );
    }
}
```

Notice what the service does:

```
text
 Business rules
     ↓
 Check duplicate email
     ↓
 Transform request
     ↓
 Hash password
     ↓
 Save entity
     ↓
 Transform entity → response DTO
```

Step 5 — Repository

```java
 @Repository
 public interface UserRepository
         extends JpaRepository<User, Long> {

     boolean existsByEmail(String email);
 }
```

Step 6 — Global exception handling

```java
 @RestControllerAdvice
 public class GlobalExceptionHandler {

     @ExceptionHandler(UserAlreadyExistsException.class)
     public ResponseEntity<ErrorResponse> handleDuplicate(
             UserAlreadyExistsException ex) {

         ErrorResponse response =
                 new ErrorResponse(
                     "USER_ALREADY_EXISTS",
                     ex.getMessage()
                 );

         return ResponseEntity
                 .status(HttpStatus.CONFLICT)
                 .body(response);
     }
 }
```

Step 7 — Successful response

```http
 HTTP/1.1 201 Created
 Content-Type: application/json
```

```json
 {
   "id": 101,
```

```
  "name": "John",
  "email": "john@example.com"
}
```

Notice:

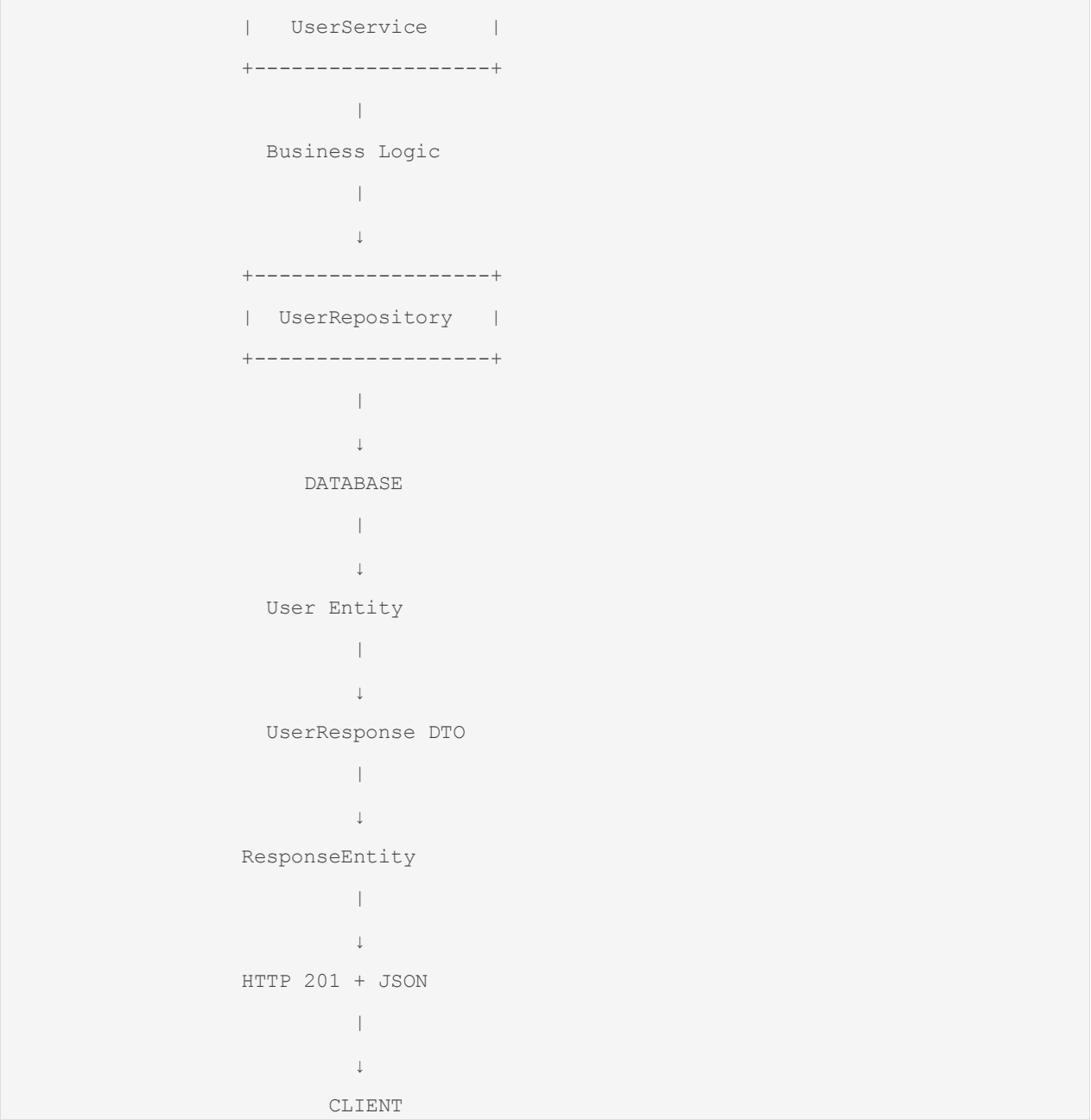
We don't return:

```
json
{
  "passwordHash": "..."
}
```

That's an internal field.

Complete architecture

```
text
                          CLIENT
                            |
                            | POST /api/v1/users
                            ↓
                  +--------------------+
                  | DispatcherServlet  |
                  +--------------------+
                            |
                            ↓
                  +--------------------+
                  | UserController     |
                  +--------------------+
                            |
                     @RequestBody
                            |
                            ↓
                    JSON → DTO
                            |
                         @Valid
                            |
                            ↓
                  +--------------------+
```



□ Part C — Follow-up questions you MUST be ready for

After you answer Q85, a product-company interviewer may immediately ask:

Follow-up 1

> *Why DTO instead of returning the Entity?*

Answer: To avoid exposing internal structure, reduce coupling, control the API contract, and separate persistence models from external representations.

Follow-up 2

> *Why put duplicate-email validation in Service instead of Controller?*

Answer:

> *Because duplicate-email checking is a business rule rather than an HTTP concern. Keeping it in the service allows the same business rule to be reused if the operation is triggered by another entry point.*

Follow-up 3

> *What happens if repository.save() throws an exception?*

This takes us directly into:

```
text
 Transactions
 Rollback
 @Transactional
```

→ Part D Q86 onward.

Follow-up 4

> *What if two requests simultaneously create the same email?*

 Very important.

Checking:

```
java
 existsByEmail(...)
```

alone is not enough.

Two requests can do:

```
text
 Request A → exists? NO
 Request B → exists? NO


 Request A → INSERT
 Request B → INSERT
```

You need a **database unique constraint** as the final protection.

For example:

```
java
 @Column(unique = true)
 private String email;
```

or preferably an explicit database unique constraint.

This is a beautiful example of why:

> *Application-level validation is not a replacement for database integrity constraints.*

□ Part C — Questions to memorize at different depths

Level 1 — Must know

```text
 Q61 DispatcherServlet
 Q62 Controller-Service-Repository
 Q63 @RequestMapping
 Q64 @GetMapping
 Q65 @PostMapping
 Q66 @PathVariable
 Q67 @RequestParam
 Q68 @RequestBody
 Q69 DTO
 Q70 Validation
 Q71 @Valid
 Q73 Exception handling
 Q74 @ControllerAdvice
 Q75 ResponseEntity
 Q76 HTTP status codes
 Q77 Idempotency
 Q78 Pagination
 Q80 API versioning
 Q82 @Controller vs @RestController
```

Level 2 — Strong 2-year candidate

Know:

```text
 DispatcherServlet request flow
 HttpMessageConverter
 Jackson
 DTO/entity separation
 401 vs 403
```

```
400 vs 404 vs 409
idempotency keys
offset vs cursor pagination
global exception response design
```

Level 3 — Product-company differentiator

Be able to reason about:

```
text
Concurrent duplicate requests
Database constraints
Large pagination
API backward compatibility
Request/response serialization
Validation vs business validation
Controller vs service responsibilities
```

□ The Part C mental model

If the interviewer suddenly asks:

> *"What happens when I call POST /api/v1/users?"*

You should immediately be able to draw:

```
text
POST /api/v1/users
        |
        ↓
   Tomcat receives
        |
        ↓
 DispatcherServlet
        |
        ↓
 HandlerMapping
        |
        ↓
 HandlerAdapter
        |
```

```
        ↓
@RequestBody
        |
        ↓
JSON → DTO
        |
        ↓
@Valid
        |
        ↓
Controller
        |
        ↓
Service
        |
        ↓
Repository
        |
        ↓
Database
        |
        ↓
Entity
        |
        ↓
DTO
        |
        ↓
HttpMessageConverter
        |
        ↓
JSON
        |
        ↓
HTTP 201
```

If you can explain that confidently, you are no longer just memorizing Spring annotations—you understand the request lifecycle.

Next → Part D: Production Spring Boot Q86–Q105

This is where the interviews become significantly more **real-world/product-company oriented**:

```
text
 @Transactional
       ↓
 Transaction Proxy
       ↓
 Transaction Manager
       ↓
 Database Connection
       ↓
 Commit / Rollback
```

We'll cover **propagation, isolation, rollback rules, self-invocation, proxy behavior, transaction boundaries, N+1 queries, connection pools, caching, logging, Actuator, health checks, and production configuration/security mistakes**.

And especially the questions interviewers love:

> *"Why didn't @Transactional work?"*
>
> *"Why doesn't @Transactional work when one method calls another method in the same class?"*
>
> *"Why is one API generating 10,000 SQL queries?"*
>
> *"What happens if 500 requests hit your service simultaneously?"*
>
> *"Why does your application become slow even though the database query itself is fast?"*
>
> *"Where exactly should you put @Transactional—Controller, Service, or Repository?"*

Those will be covered from **Q86 onward** with the same **short answer → internals → project example → follow-up → tricky counter-question** format.

you asked: ok continu

message time: 2026-08-23 03:33:23

ok continue as aboved part D .

Part D — Production Spring Boot

Q86–105

- Transactions

- `@Transactional`
- propagation
- isolation
- rollback
- proxy behavior
- self-invocation
- transaction boundaries
- N+1
- connection pools
- caching
- logging
- Actuator
- health checks
- configuration/security mistakes

Part E — Actual Interview Scenarios

Q106–120

These will be the **most valuable**.

For example:

> *"Your API is suddenly taking 5 seconds instead of 200 ms. How do you investigate?"*
>
> *"You added @Transactional, but rollback isn't happening. Why?"*
>
> *"Two beans implement the same interface. What happens?"*
>
> *"Your application starts locally but fails in production. How do you debug it?"*
>
> *"Why does adding spring-boot-starter-data-jpa suddenly create database-related infrastructure?"*
>
> *"Your API returns 500 for invalid input. How would you redesign the exception handling?"*
>
> *"Your Spring Boot service has 10,000 DB queries for one API request. What's happening?"*

These are the questions that turn **memorization into interview performance**.

Recent 2-year interview guidance specifically recommends being able to build a small end-to-end feature—**controller → service → repository → DTO → validation → exception handling**—and explain the design decisions, rather than only knowing annotation definitions.

chatgpt response: Part D is

Part D is where we move from framework knowledge into **production behavior**: transaction boundaries, proxy traps, database performance, connection pools, caching, observability, and security mistakes.

Module 2 — Spring & Spring Boot

Part D — Production Spring Boot

Q86–Q105

This part is where interviewers usually stop asking:

> *"What is this annotation?"*

and start asking:

> *"Your production system is failing. What is happening internally, and how would you fix it?"*

The most important mental model here is:

```
text
                      Production API
                           |
         +-----------------+-----------------+
         |                 |                 |
         ↓                 ↓                 ↓
    Transaction         Database          Cache
         |                 |                 |
         ↓                 ↓                 ↓
     Proxy/AOP       Connection Pool     Redis/Caffeine
         |                 |
         ↓                 ↓
    Commit/Rollback      SQL
         |
         ↓
     Observability
         |
    Logs / Metrics /
  Health / Traces
```

Q86. What is a Transaction?

⭐ Short interview answer

> *A transaction is a unit of work that should be treated as one logical operation, with guarantees about consistency and atomicity. If the transaction succeeds, its changes are committed; if it fails according to the transaction rules, its changes can be rolled back.*

Classic example:

```
text
 Transfer ₹100 from A → B
```

We need:

```
text
 A balance -= 100
 B balance += 100
```

Not:

```
text
 A balance -= 100
 B update FAILED
```

Otherwise money disappears.

So:

```
text
 Transaction
    |
    +-- Debit A
    |
    +-- Credit B
    |
    +-- Commit
```

or:

```
text
 Transaction
    |
    +-- Debit A
    |
    +-- Credit B FAILED
```

```
    |
    +-- Rollback
```

Spring provides a consistent abstraction over transaction technologies including JDBC, JPA, Hibernate, and JTA. (Home)

Q86.1 What are ACID properties?

A — Atomicity

All operations in the transaction succeed or the transaction is rolled back.

```
text
 A + B + C
```

Either:

```
text
 A + B + C committed
```

or:

```
text
 nothing committed
```

C — Consistency

The database should move from one valid state to another valid state while respecting its constraints/invariants.

I — Isolation

Concurrent transactions should not interfere in invalid ways.

We'll go deep into this in **Q91**.

D — Durability

Once committed, changes should survive appropriate system failures.

Q87. What is `@Transactional`?

 One of the highest-value Spring interview questions.

⭐ Short interview answer

> *@Transactional declares that a method or class should execute with transactional semantics. In typical Spring applications, transaction interception is applied around the method call,*

> *allowing Spring to begin, commit, or roll back the transaction according to the configured rules.*

Example:

```java
 @Transactional
 public void transferMoney(
         Long fromAccount,
         Long toAccount,
         BigDecimal amount) {

     debit(fromAccount, amount);
     credit(toAccount, amount);
 }
```

Conceptually:

```text
 Caller
   ↓
 Transactional Proxy
   ↓
 Begin Transaction
   ↓
 transferMoney()
   ↓
 Success?
  /      \
 YES      NO
  ↓        ↓
 Commit   Rollback
```

Spring's declarative transaction support is implemented using Spring AOP, and the transaction interceptor handles the transactional method invocation. (Home)

Q87.1 Where should `@Transactional` normally be placed?

⭐ Best answer

Usually on the **service-layer method** that represents a business transaction.

Example:

```java
@Service
public class OrderService {

    @Transactional
    public void placeOrder(OrderRequest request) {

        createOrder();
        reserveInventory();
        createPaymentRecord();
    }
}
```

Why?

Because the service layer usually defines the business operation.

You generally don't want:

```text
Controller
   ↓
transaction
   ↓
Service
   ↓
Repository
```

where the transaction boundary is determined by HTTP handling.

Instead:

```text
Controller
   ↓
Service
   ↓
@Transactional
   ↓
Repository
```

Q87.2 Should you put `@Transactional` on a Repository?

It can be used there, but that isn't usually the primary business transaction boundary.

Imagine:

```text
createOrder()
  ├── saveOrder()
  ├── reserveInventory()
  ├── createPayment()
  └── publish event
```

The transaction should typically encompass the **business operation**, not just one repository call.

So:

```text
Service method
       ↓
transaction boundary
       |
       +-- repository 1
       +-- repository 2
       +-- repository 3
```

That's why service-level transaction boundaries are common.

Q88. What happens internally when `@Transactional` is used?

 Very important.

Suppose:

```java
@Service
public class OrderService {

    @Transactional
    public void placeOrder() {
        ...
    }
}
```

Spring can create a proxy around the target bean.

```text
 Application
      |
      ↓
 OrderService Proxy
      |
      | transaction interceptor
      ↓
 OrderService target
      |
      ↓
 placeOrder()
```

The proxy can:

```text
 1. Detect transaction metadata
 2. Ask transaction manager to start/join transaction
 3. Invoke target method
 4. Detect success/failure
 5. Commit or roll back
```

Spring's `TransactionInterceptor` is the core interceptor responsible for applying transactional behavior. (Home)

Q88.1 What is `PlatformTransactionManager`?

⭐ Interview answer

> *PlatformTransactionManager is Spring's abstraction for transaction management. The concrete implementation coordinates transactions for technologies such as JDBC or JPA.*

Conceptually:

```text
 @Transactional
       ↓
 TransactionInterceptor
       ↓
 PlatformTransactionManager
```

```
      ↓
JPA / JDBC / Hibernate transaction
```

Depending on the application's setup, a different transaction manager can be used.

Q88.2 What is actually being committed?

This depends on the underlying resource/transaction system.

For a typical database application:

```text
Spring transaction
       ↓
Database transaction
       ↓
Connection
       ↓
COMMIT / ROLLBACK
```

Spring provides the abstraction and coordinates the transactional work; it isn't itself a database.

Q89. What is transaction propagation?

□□ **Very common product-company question.**

⭐ Short interview answer

> *Propagation defines how a method's transaction should behave when it is called while another transaction already exists. Should it join the existing transaction, create a new one, suspend the existing one, or use some other behavior?*

Common modes:

```text
REQUIRED
REQUIRES_NEW
SUPPORTS
MANDATORY
NOT_SUPPORTED
NEVER
NESTED
```

The two you should know extremely well:

```text
REQUIRED
REQUIRES_NEW
```

Q89.1 `PROPAGATION_REQUIRED`

Meaning

> *Join the existing transaction if one exists; otherwise create a new transaction.*

Example:

```java
@Transactional
public void methodA() {
    methodB();
}
```

and:

```java
@Transactional(
    propagation = Propagation.REQUIRED
)
public void methodB() {
}
```

Conceptually:

```text
methodA
  ↓
Transaction T1
  ↓
methodB
  ↓
join T1
```

Spring's current default propagation for `@Transactional` is `PROPAGATION_REQUIRED`. (Home)

Q89.2 `REQUIRES_NEW`

Meaning

> *Always use a new transaction. If an existing transaction exists, suspend it while the new transaction executes.*

Conceptually:

```
text
 Outer Transaction T1
        |
        ↓
    methodB()
        |
        ↓
 Suspend T1
        |
        ↓
 Create T2
        |
        ↓
 methodB completes
        |
        ↓
 Commit/Rollback T2
        |
        ↓
 Resume T1
```

Q89.3 Real-world example for `REQUIRES_NEW`

Suppose you have:

```
text
 Order transaction
     ↓
 Save order
     ↓
 Save audit
```

You might want an audit record to be committed independently in a specific design.

```java
@Transactional
public void placeOrder() {

    saveOrder();

    auditService.saveAudit();
}
```

and:

```java
@Transactional(
    propagation = Propagation.REQUIRES_NEW
)
public void saveAudit() {
    ...
}
```

Now:

```text
T1 → Order
 |
 +-- suspend
 |
 T2 → Audit
 |
 +-- commit
 |
 resume T1
```

But be careful:

> *REQUIRES_NEW is not a magical "make this survive anything" switch.*

Its semantics and failure behavior must be considered carefully.

Q89.4 When would `REQUIRES_NEW` be dangerous?

Because every new transaction can require another database connection while the outer transaction's connection remains involved.

With many concurrent requests:

```
text
 Request 1
   T1 → connection A
   T2 → connection B

 Request 2
   T1 → connection C
   T2 → connection D

 ...
```

A poorly designed use of `REQUIRES_NEW` can increase connection demand and contribute to pool exhaustion.

That's a very good production-level observation.

Q90. What are other propagation modes?

You should at least know the meaning:

Propagation	Meaning
`REQUIRED`	Join existing or create new
`REQUIRES_NEW`	Always create new, suspend existing
`SUPPORTS`	Use existing if available, otherwise run without one
`MANDATORY`	Must already have a transaction
`NOT_SUPPORTED`	Run without transaction, suspending existing
`NEVER`	Fail if transaction exists
`NESTED`	Nested transaction semantics where supported

For a 2-year interview:

> *Master REQUIRED and REQUIRES_NEW; understand the intent of the others.*

Q91. What is transaction isolation?

□□ **Very important database + Spring question.**

⭐ Short answer

> *Isolation determines how concurrent transactions can observe each other's changes and helps control anomalies such as dirty reads, non-repeatable reads, and phantom reads.*

Common levels:

```text
 READ_UNCOMMITTED
 READ_COMMITTED
 REPEATABLE_READ
 SERIALIZABLE
```

Spring exposes these through:

```java
 @Transactional(
     isolation = Isolation.READ_COMMITTED
 )
```

Spring documents `ISOLATION_DEFAULT` as the default setting, meaning the underlying transaction system/database determines the actual isolation level. (Home)

Q91.1 Dirty Read

Transaction A changes data:

```text
 A:
 balance = 500
       ↓
 changes to 300
       ↓
 not committed
```

Transaction B reads:

```text
 balance = 300
```

Then A rolls back.

Actual database value returns to:

```text
 500
```

B read data that was never committed.

That's a:

> *Dirty read*

Q91.2 Non-repeatable Read

Transaction A:

```
text
 SELECT balance
 → 500
```

Transaction B updates:

```
text
 500 → 300
 COMMIT
```

Transaction A reads again:

```
text
 SELECT balance
 → 300
```

Same transaction, same row, different value.

That's:

> *Non-repeatable read*

Q91.3 Phantom Read

Transaction A:

```
sql
 SELECT *
 FROM users
 WHERE age > 30;
```

Returns:

```
text
 10 rows
```

Transaction B inserts another matching user.

Transaction A runs the query again:

```
text
 11 rows
```

A new matching row appeared.

That's:

> *Phantom read*

Q91.4 Isolation levels — interview view

Conceptually:

```
text
 READ_UNCOMMITTED
     ↓
 weakest isolation
     ↓
 READ_COMMITTED
     ↓
 REPEATABLE_READ
     ↓
 SERIALIZABLE
     ↓
 strongest / most restrictive
```

The tradeoff is roughly:

```
text
 More isolation
       ↓
 more concurrency restrictions / potential cost

 Less isolation
       ↓
 more concurrency
 but more anomalies
```

Exact behavior also depends on the database implementation.

Q92. What is rollback?

⭐ Short answer

> *Rollback means undoing the changes made by the current transaction instead of committing them.*

Example:

```text
 Transaction
    ↓
 Insert Order
    ↓
 Update Inventory
    ↓
 Payment operation fails
    ↓
 Rollback
    ↓
 Undo transaction changes
```

Q92.1 What causes rollback with `@Transactional`?

This is a huge interview trap.

By default, Spring's declarative transaction behavior rolls back for:

```text
 RuntimeException
 Error
```

but not automatically for ordinary checked exceptions. (Home)

Example:

```java
 @Transactional
 public void createOrder() {

     saveOrder();
```

```
    throw new RuntimeException();
}
```

Typically:

```
text
RuntimeException
      ↓
Rollback
```

But:

```
java
@Transactional
public void createOrder()
        throws Exception {

    saveOrder();

    throw new Exception();
}
```

By default, a checked `Exception` does not trigger the same automatic rollback rule.

Q92.2 How do you rollback for checked exception?

Use:

```
java
@Transactional(
    rollbackFor = Exception.class
)
public void createOrder() throws Exception {
    ...
}
```

Then the specified checked exception can trigger rollback.

Current Spring documentation also notes that the default rollback behavior can be globally customized in modern Spring versions; nevertheless, knowing the traditional `@Transactional` defaults remains essential for interviews. (Home)

Q92.3 Tricky question

> *"If an exception happens inside a transaction, does rollback always happen?"*

❌No.

You must consider:

```
text
 Exception type
 Rollback rules
 Transaction boundary
 Proxy interception
 Whether exception escapes the transactional method
 Transaction manager/resource
```

For example:

```
java
 @Transactional
 public void saveData() {

     try {
         repository.save(...);

         throw new RuntimeException();

     } catch (RuntimeException e) {

         // swallowed
     }
 }
```

The exception may not reach the transaction interceptor in the way required to trigger the expected rollback.

This leads to an important production debugging issue.

Q93. Why does `@Transactional` sometimes not work?

□□□ **One of the most important interview questions.**

Potential causes include:

```text
1. Self-invocation
2. Method isn't invoked through the Spring proxy
3. Object created using new
4. Wrong transaction manager/configuration
5. Exception isn't covered by rollback rules
6. Transaction boundary is elsewhere
7. Method visibility/proxy limitations
```

Let's examine the biggest one.

Q93.1 What is self-invocation?

Suppose:

```java
@Service
public class OrderService {

    public void placeOrder() {
        saveOrder();
    }

    @Transactional
    public void saveOrder() {
        ...
    }
}
```

You might expect:

```text
placeOrder()
   ↓
transaction starts
   ↓
saveOrder()
```

But the call is:

```java
this.saveOrder();
```

So it doesn't go through the Spring proxy.

Spring's AOP is proxy-based, and calls made directly through `this` bypass the proxy/advice. (Home)

Q93.2 The proxy picture

Actual Spring-managed reference:

```text
Caller
  |
  ↓
Proxy
  |
  ↓
Target Object
```

External call:

```text
caller → proxy → transaction advice → target
```

Self-call:

```text
target
  |
  ↓
this.saveOrder()
  |
  ↓
target directly
```

The proxy is bypassed.

Therefore:

```text
@Transactional
```

may not be applied to that self-invocation.

Q93.3 How do you fix self-invocation?

⭐ Preferred

Refactor into another bean:

```
text
 OrderService
      |
      ↓
 OrderTransactionService
      |
      ↓
 @Transactional method
```

Now the call travels through the Spring proxy.

Spring's own documentation recommends avoiding self-invocation where possible. (Home)

Other approaches exist, such as self-injection or `AopContext.currentProxy()`, but `AopContext.currentProxy()` tightly couples the code to Spring AOP and is generally discouraged. (Home)

Q94. What are Spring AOP proxy types?

⭐ Interview answer

Spring AOP can use:

```
text
 JDK dynamic proxy
 CGLIB/class-based proxy
```

A JDK dynamic proxy works through interfaces.

A class-based proxy uses subclassing.

Spring's current documentation describes these mechanisms and the limitations of class-based proxies, including restrictions involving `final` classes/methods and private methods. (Home)

Q94.1 Why should I care about proxies?

Because many Spring features depend on interception.

For example:

```text
@Transactional
@Cacheable
@Async
AOP advice
```

Conceptually:

```text
Your Bean
   ↑
Proxy
   ↑
Spring behavior
```

Therefore, understanding:

```text
"Is this method call going through the proxy?"
```

can solve many mysterious Spring issues.

Q95. What should the transaction boundary be?

⭐ Interview answer

> *The transaction boundary should normally match a meaningful business operation that must be atomic. In a layered application this usually means a service-layer method coordinating one business use case.*

Example:

```java
@Transactional
public void checkout(CheckoutRequest request) {

    validateCart();

    createOrder();

    reserveInventory();
```

```
    createPaymentRecord();
}
```

The boundary is:

```text
BEGIN
 |
 +-- validate
 +-- create order
 +-- reserve inventory
 +-- create payment
 |
COMMIT
```

or:

```text
BEGIN
 |
 +-- operation fails
 |
ROLLBACK
```

Q95.1 Bad transaction boundary

Imagine:

```java
@Transactional
public void entireHttpRequest() {
    // call slow external API
    // perform DB operations
    // wait 5 seconds
}
```

Now the database transaction might remain active while waiting on an external service.

That can cause:

```text
Connection held
```

```
   ↓
Transaction remains open
   ↓
More concurrent requests
   ↓
Pool pressure
   ↓
Performance degradation
```

A strong production principle is:

> *Keep transactions focused and short, especially when external network calls are involved.*

But don't artificially split a business operation that genuinely needs atomicity.

Q96. What is the N+1 Query Problem?

□□□ **Extremely important for Java/Spring/JPA interviews.**

⭐ Short answer

> *The N+1 problem occurs when an initial query loads N parent records and then the application performs an additional query for each parent, resulting in 1 + N database queries.*

Example:

```text
Query 1:
SELECT * FROM orders;

Then:

Query 2:
SELECT * FROM order_items WHERE order_id = 1;

Query 3:
SELECT * FROM order_items WHERE order_id = 2;

Query 4:
SELECT * FROM order_items WHERE order_id = 3;

...
```

For 1,000 orders:

```
text
 1 + 1000 = 1001 queries
```

Q96.1 Why is N+1 bad?

Because database round trips are expensive.

Instead of:

```
text
 1 query
```

you have:

```
text
 1001 queries
```

Potential impact:

```
text
 DB CPU ↑
 Network traffic ↑
 Latency ↑
 Connection usage ↑
 Application response time ↑
```

Q96.2 Common causes

With JPA/Hibernate, N+1 can occur because of:

```
text
 Lazy associations accessed inside loops
 Entity traversal
 Poor fetch strategy
 Serialization touching relationships
```

Example:

```
java
 List<Order> orders = repository.findAll();
```

```
for (Order order : orders) {
    System.out.println(
        order.getItems().size()
    );
}
```

This may trigger additional queries for the items.

Q96.3 How do you solve N+1?

Several strategies depending on the use case:

Fetch join

```jpql
SELECT o
FROM Order o
JOIN FETCH o.items
```

Entity graph

```java
@EntityGraph(attributePaths = "items")
List<Order> findAll();
```

DTO projection

Return exactly what the API needs.

Batch fetching

Configure batching where appropriate.

But:

> *Don't blindly make everything EAGER.*

You can simply replace one problem with:

```text
Huge joins
Huge result sets
High memory
Slow queries
```

Q96.4 How would you detect N+1 in production?

A strong answer:

```
text
 Enable SQL logging carefully in non-production/debug environments
         ↓
 Use Hibernate statistics / metrics
         ↓
 Inspect APM traces
         ↓
 Look for repeated SQL patterns
         ↓
 Measure query count per request
```

A particularly useful test is:

> *"How many SQL statements does this API execute for 20 records?"*

If the answer unexpectedly becomes:

```
text
 21
```

you may have N+1.

Q97. What is a Connection Pool?

⭐ Short interview answer

> *A connection pool maintains a reusable set of database connections so the application doesn't have to establish a new physical database connection for every request.*

Without pooling:

```
text
 HTTP request
    ↓
 Create DB connection
    ↓
 Query
    ↓
 Close connection
```

Every request pays connection-establishment cost.

With pooling:

```
text
 Application
     |
     ↓
 Connection Pool
     |
  +--+--+--+--+
  |  |  |  |  |
  C1 C2 C3 C4 C5
```

A request borrows one:

```
text
 Request
   ↓
 borrow C2
   ↓
 execute query
   ↓
 return C2
```

Q97.1 What is HikariCP?

In typical Spring Boot SQL/JPA applications, HikariCP is commonly selected as the connection pool when available. Spring Boot's current documentation states that HikariCP is preferred and that the JDBC/JPA starters bring it in automatically. (Home)

Conceptually:

```
text
 Spring Boot
     ↓
 DataSource
     ↓
 HikariCP
     ↓
 DB connections
```

Q97.2 What happens if the pool is exhausted?

Suppose:

```
text
 Maximum connections = 10
```

and 10 requests currently hold connections:

```
text
 C1 C2 C3 C4 C5 C6 C7 C8 C9 C10
```

Request #11 wants a connection:

```
text
 Request #11
      ↓
 Pool
      ↓
 No connection available
      ↓
 Wait
```

If no connection becomes available within the configured timeout:

```
text
 Connection acquisition timeout
```

Now API latency rises.

This can create a chain reaction:

```
text
 Slow queries
     ↓
 Connections held longer
     ↓
 Pool exhaustion
     ↓
 Requests wait
     ↓
 More requests pile up
```

```
    ↓
System appears "down"
```

Q97.3 Does increasing pool size always fix performance?

❌Absolutely not.

Suppose database can comfortably handle:

```
text
20 concurrent DB operations
```

You configure:

```
text
500 connections
```

Now:

```
text
500 application threads
        ↓
many simultaneous DB operations
        ↓
DB overload
        ↓
query latency increases
        ↓
connections remain occupied longer
```

A larger pool can make overload worse.

Pool sizing should account for:

```
text
Database capacity
Application concurrency
Query duration
Number of application instances
CPU
Connection limits
Workload
```

Q98. What is caching?

⭐ Short interview answer

> *Caching stores frequently accessed data in a faster storage layer so that repeated requests can avoid expensive computation or database access.*

Without cache:

```
text
 Request
   ↓
 Application
   ↓
 Database
   ↓
 Result
```

With cache:

```
text
 Request
   ↓
 Cache
   |
   +-- HIT → return immediately
   |
   +-- MISS → DB → cache → return
```

Q98.1 What is Spring Cache Abstraction?

Spring provides annotations such as:

```
text
 @Cacheable
 @CachePut
 @CacheEvict
 @CacheConfig
```

Spring's cache abstraction provides a consistent programming model but does not itself constitute a complete distributed cache implementation. A concrete cache provider/store must back it. (Home)

Q98.2 What is `@Cacheable`?

⭐ Interview answer

> *@Cacheable stores the result of a method invocation and can return the cached value on subsequent calls instead of executing the method again.*

Example:

```java
 @Cacheable("users")
 public UserResponse getUser(Long id) {


     return repository.findById(id)
             .map(this::toResponse)
             .orElseThrow();
 }
```

First call:

```text
 getUser(101)
     ↓
 Cache miss
     ↓
 Database
     ↓
 Result
     ↓
 Cache
```

Second call:

```text
 getUser(101)
     ↓
 Cache hit
     ↓
 Return cached result
```

Spring's cache documentation describes this exact behavior. (Home)

Q98.3 What is `@CacheEvict`?

Used to remove stale data.

Suppose:

```java
@Cacheable("users")
public UserResponse getUser(Long id) {
    ...
}
```

Then:

```java
@CacheEvict(
    value = "users",
    key = "#id"
)
public void updateUser(Long id, UpdateRequest request) {
    ...
}
```

Conceptually:

```text
Read
 ↓
Cache

Update
 ↓
DB
 ↓
Evict cache
```

Otherwise:

```text
DB → John Smith
Cache → John
```

```
API reads cache
→ stale John
```

Spring provides explicit eviction operations for this purpose. (Home)

Q98.4 Cache problem: stale data

This is one of the most important production concerns.

Suppose:

```
text
 DB:
 balance = 1000

 Cache:
 balance = 1000
```

Database changes:

```
text
 DB:
 balance = 500
```

but cache still contains:

```
text
 1000
```

Now the application may return stale data.

So caching introduces a consistency problem.

Q99. What is Cache-Aside pattern?

A common production approach:

```
text
 READ

 Application
    ↓
 Check cache
```

```
    |
  +--+--+
  |     |
 Hit   Miss
  |     |
  ↓     ↓
 Return DB
        ↓
      Update cache
        ↓
      Return
```

Write:

```text
 Application
     ↓
 Update DB
     ↓
 Invalidate cache
```

Conceptually:

```text
 DB = source of truth
 Cache = performance layer
```

This is a very useful interview answer.

Q99.1 What are cache dangers?

Don't say:

> *"Caching always improves performance."*

Potential problems:

```text
 Stale data
 Cache stampede
 Memory growth
 Wrong cache keys
```

```
Invalidation bugs
Serialization issues
Distributed consistency
Hot keys
Cold cache after restart
```

The classic statement:

> *"There are only two hard things in Computer Science: cache invalidation and naming things."*

You don't need to quote it in an interview; just understand that invalidation is genuinely difficult.

Q100. What is logging?

⭐ Short interview answer

> *Logging records application and operational events so that developers and operators can understand system behavior, diagnose failures, and investigate production incidents.*

Typical levels:

```text
TRACE
DEBUG
INFO
WARN
ERROR
```

Q100.1 What should you log?

Useful:

```text
Request identifiers
Important business events
External call failures
Unexpected exceptions
State transitions
Latency-related information
Security-relevant events
```

But be careful.

Do **not** log:

```text
 Passwords
 Access tokens
 Secrets
 Full credit-card numbers
 Sensitive personal data
```

Q100.2 What is structured logging?

Instead of:

```text
 Order created successfully for user 101
```

structured logs might contain:

```json
 {
   "event": "order_created",
   "orderId": "5001",
   "userId": "101",
   "durationMs": 120
 }
```

This is easier for log aggregation/search systems to query.

Q100.3 What is a correlation ID / trace ID?

Suppose:

```text
 Client
   ↓
 API Gateway
   ↓
 Order Service
   ↓
 Payment Service
```

```
  ↓
 Inventory Service
```

One user request could generate dozens of log lines.

A request/trace ID lets you correlate them:

```
text
 traceId = abc123
```

Then:

```
text
 Order Service   abc123
 Payment Service abc123
 Inventory       abc123
```

Now you can follow one request across services.

Q101. What is Actuator used for in production?

We discussed basic Actuator earlier, but now think operationally.

It can expose operational endpoints for:

```
text
 health
 metrics
 info
 beans
 loggers
```

depending on exposure/configuration.

Example:

```
text
 /actuator/health
 /actuator/metrics
```

The exact endpoints you expose should be intentional and secured appropriately.

Q101.1 Why should Actuator endpoints not all be publicly exposed?

Suppose you expose sensitive operational information publicly.

An attacker might learn:

```text
 Internal configuration
 Environment information
 Bean information
 Metrics
 Application structure
```

That can increase attack surface.

Production principle:

```text
 Actuator
    ↓
 Secure
    ↓
 Expose only what is necessary
    ↓
 Restrict access
```

Q102. What are liveness and readiness health checks?

⭐ Interview answer

Liveness asks:

> *"Should this process remain alive?"*

Readiness asks:

> *"Should this instance receive traffic?"*

Spring Boot Actuator exposes dedicated Kubernetes-oriented probe groups:

```text
 /actuator/health/liveness
 /actuator/health/readiness
```

Current Spring Boot documentation explicitly warns that liveness should generally not depend on external systems such as databases or external APIs, because an external outage could otherwise cause all instances to restart and create a cascading failure. (Home)

Q102.1 Real-world example

Suppose:

```
text
 Application starts
```

but:

```
text
 Database connection not ready
```

You might want:

```
text
 Liveness = UP
 Readiness = DOWN
```

Meaning:

```
text
 Process is alive
 but
 don't send traffic yet
```

Once dependencies and startup tasks are ready:

```
text
 Liveness = UP
 Readiness = UP
```

Q103. What are common Spring Boot configuration mistakes?

 Production interview question.

Examples:

1. Wrong profile

```
text
 DEV profile accidentally active in PROD
```

2. Wrong database URL

```text
 application-prod.properties
```

points to incorrect DB.

3. Missing environment variable

```text
 PAYMENT_API_URL
```

not configured.

4. Configuration precedence mistake

An environment variable or command-line argument overrides the value you expected.

5. Hardcoded secrets

```properties
 password=secret
```

inside source control.

6. Unsafe actuator exposure

Too much operational information exposed.

7. Excessive logging

Sensitive data or huge production volume.

Q103.1 What is the debugging approach?

Don't randomly change annotations.

Use:

```text
 1. Check active profile
 2. Check effective configuration
 3. Check environment variables
 4. Check startup logs
 5. Check stack trace/root cause
 6. Check bean creation
 7. Check auto-configuration conditions
 8. Check external dependencies
```

```
9. Check health endpoints
10. Reproduce with minimal change
```

The interviewer wants **methodical debugging**, not guesswork.

Q104. What are common Spring Security mistakes?

Even before the dedicated Security module, you should recognize these:

Mistake 1 — Trusting client input

Never assume:

```
json
{
  "role": "ADMIN"
}
```

means the user is actually admin.

Authorization must come from trusted security context/server-side rules.

Mistake 2 — Logging JWT/access tokens

Bad:

```
text
Authorization: Bearer eyJ...
```

in logs.

Mistake 3 — Exposing sensitive actuator endpoints

As discussed.

Mistake 4 — Returning internal exceptions

Bad:

```
json
{
  "error": "org.hibernate.exception.SQLGrammarException..."
}
```

Clients shouldn't receive your internal implementation details.

Mistake 5 — Missing authorization checks

Authentication:

> *Who are you?*

Authorization:

> *Are you allowed to do this?*

Don't confuse them.

Q104.1 IDOR / insecure object access example

Suppose:

```http
GET /api/accounts/101
```

A user changes it to:

```http
GET /api/accounts/102
```

If your controller simply does:

```java
accountRepository.findById(id)
```

without checking ownership/authorization, the user may access another user's account.

That's a serious authorization flaw.

The interviewer may not expect the acronym, but the reasoning is important:

> *Never assume that possession of a resource ID means permission to access that resource.*

Q105. Your API is slow. What production metrics would you inspect?

□□ **This is the bridge to Part E.**

Suppose:

```text
API response time:
200 ms → 5 seconds
```

Don't immediately say:

> *"Optimize the code."*

Start with observability.

Check:

```
text
                      API latency
                          |
        +-----------------+------------------+
        |                 |                  |
        ↓                 ↓                  ↓
    Application         Database          External
      timing             timing            services
        |                 |                  |
        ↓                 ↓                  ↓
  CPU / GC /          Query time         Network
  thread pool         query count       timeout
        |
        ↓
  connection pool
        |
        ↓
  pool wait time
```

Q105.1 What would you specifically investigate?

1. Request latency

```
text
 p50
 p95
 p99
```

Don't only look at average latency.

Example:

```
text
 Average = 200 ms
 p99 = 7 seconds
```

The average can hide severe tail latency.

2. Database

Check:

```
text
 Query latency
 Query count
 Slow queries
 N+1
 Lock contention
 Connection pool wait
```

3. JVM/application

Check:

```
text
 CPU
 Heap
 GC
 Thread pools
 Thread contention
```

4. External dependencies

Check:

```
text
 HTTP latency
 Timeouts
 Retries
 5xx rates
 Connection pools
```

5. Cache

Check:

```
text
 Hit rate
 Miss rate
 Eviction
 Latency
 Hot keys
```

□ Q105.2 Production diagnosis example

Suppose your API normally takes:

```
text
 200 ms
```

Now it takes:

```
text
 5 seconds
```

You inspect traces:

```
text
 Controller:       5 ms
 Service:          10 ms
 Database:         4,850 ms
 JSON serialization: 15 ms
```

Now database is clearly the bottleneck.

Then DB metrics show:

```
text
 Query count = 1001
```

You identify:

```
text
 N+1 query problem
```

You fix it using:

```
text
 fetch join
 projection
 @EntityGraph
 batching
```

and response becomes:

```
text
 200 ms
```

This is much stronger than saying:

> *"I would increase server resources."*

□ Part D — The Most Important Concepts

At this point you should be able to explain:

```
text
 @Transactional
      ↓
 Proxy
      ↓
 TransactionInterceptor
      ↓
 TransactionManager
      ↓
 DB transaction
      ↓
 Commit / Rollback
```

and:

```
text
 API
  ↓
 Service
  ↓
 Repository
  ↓
 Connection Pool
  ↓
 Database
```

and:

```
text
 API
  ↓
 Cache
  |
  +-- HIT → return
```

```
  |
  +-- MISS → DB → cache → return
```

and:

```
text
 Production
    |
    +-- Logs
    +-- Metrics
    +-- Health
    +-- Traces
    +-- Alerts
```

□ The 15 Part-D questions I would prioritize

Priority	Question	Why
⋆ ⋆ ⋆	Q86 Transaction	Foundation
⋆ ⋆ ⋆	Q87 `@Transactional`	Extremely common
⋆ ⋆ ⋆	Q88 How `@Transactional` works internally	Differentiator
⋆ ⋆ ⋆	Q89 Propagation	Very common
⋆ ⋆ ⋆	Q91 Isolation	Database depth
⋆ ⋆ ⋆	Q92 Rollback	Common trap
⋆ ⋆ ⋆	Q93 Why `@Transactional` doesn't work	Excellent interview question
⋆ ⋆ ⋆	Q94 Proxy behavior	Core Spring internals
⋆ ⋆ ⋆	Q95 Transaction boundaries	Architecture
⋆ ⋆ ⋆	Q96 N+1	Extremely common JPA issue
⋆ ⋆ ⋆	Q97 Connection pools	Production
⋆ ⋆	Q98 Caching	Production
⋆ ⋆ ⋆	Q99 Cache consistency	Product-company depth
⋆ ⋆ ⋆	Q102 Liveness/readiness	Cloud/Kubernetes
⋆ ⋆ ⋆	Q105 Production performance diagnosis	Real-world debugging

□ A 2-Year Interview "Golden Chain"

When an interviewer asks:

> *"Why is my API slow?"*

Your reasoning should automatically go:

```
text
 API slow
    ↓
 Measure first
    ↓
 p50/p95/p99
    ↓
 Distributed trace
    ↓
 Application timing
    ↓
 DB timing
    ↓
 Query count
    ↓
 N+1?
    ↓
 Slow query?
    ↓
 Connection pool?
    ↓
 External API?
    ↓
 Cache hit rate?
    ↓
 CPU / GC / threads?
```

When they ask:

> *"Why didn't my transaction rollback?"*

Think:

```
text
 @Transactional
    ↓
 Was call through proxy?
    ↓
 Self invocation?
    ↓
 Object created with new?
    ↓
 Exception type?
    ↓
 rollbackFor?
    ↓
 Was exception swallowed?
    ↓
 Correct transaction manager?
    ↓
 Correct transaction boundary?
```

That's the level of reasoning we're building.

□ One Combined Production Scenario

Interviewer:

> *"You have an Order API. It used to respond in 200 ms, but now takes 5 seconds. At the same time, database CPU increased and connection-pool usage is near maximum. How would you investigate?"*

⭑ Strong 2-year answer

> *"I would first confirm the latency increase using p95/p99 rather than only averages. Then I'd inspect distributed traces and database metrics to determine whether the time is spent in application logic, database calls, or external dependencies. Since database CPU and connection-pool usage are high, I'd inspect SQL query count and duration, looking for N+1 queries or newly introduced expensive queries. I'd also check whether transactions are held open longer than expected and whether slow external calls are occurring inside transaction boundaries. Then I'd inspect connection-pool wait time and sizing. If I find N+1, I'd fix the data-fetch strategy rather than simply increasing the pool. Finally I'd verify the improvement using the same latency and query-count metrics."*

Notice the reasoning:

```
text
 Observe
   ↓
 Measure
   ↓
 Localize
   ↓
 Find root cause
   ↓
 Fix root cause
   ↓
 Measure again
```

That is exactly how you should answer production troubleshooting questions.

Part D complete

You now have the foundation for:

```
text
 Part A — Spring Core
 Q1–30

 Part B — Spring Boot Internals
 Q31–60

 Part C — Spring MVC + REST
 Q61–85

 Part D — Production Spring Boot
 Q86–105
```

The remaining **Q106–Q120 — Part E** will be deliberately different. They won't mainly be definition questions.

They will be **mock product-company interview scenarios**, where the interviewer gives you a production problem and keeps asking follow-ups until they expose whether you actually understand Spring Boot.

Examples:

```
text
 Q106 — API latency suddenly increases from 200 ms → 5 sec
 Q107 — @Transactional rollback isn't working
 Q108 — Self-invocation breaks transaction
 Q109 — Two beans implement same interface
 Q110 — Works locally, fails in production
 Q111 — spring-boot-starter-data-jpa creates unexpected infrastructure
 Q112 — Invalid request returns 500
 Q113 — 10,000 queries generated for one request
 Q114 — Connection pool exhausted
 Q115 — Cache returns stale data
 Q116 — API works but Kubernetes keeps restarting it
 Q117 — Memory/CPU suddenly increases
 Q118 — External payment API is slow
 Q119 — Design a production-ready Order API
 Q120 — Full end-to-end Spring Boot debugging interview
```

For each one, I'll format the reasoning as:

```
text
 Scenario
    ↓
 What interviewer is testing
    ↓
 First answer
    ↓
 Deep investigation
    ↓
 Likely root causes
    ↓
 Fix
    ↓
 Follow-up questions
    ↓
 Tricky counter-question
    ↓
 Best final answer
```

That will complete **Module 2 — all 120 Spring/Spring Boot questions** before we move to **Module 3 — JPA/Hibernate 80 questions**.

you asked: ok continu

message time: 2026-08-23 03:37:07

ok continue as aboved part e .

Part B — Spring Boot Internals

Q31–60

- Spring vs Spring Boot
- `@SpringBootApplication`
- auto-configuration
- starters
- conditional configuration
- `@ConditionalOnClass`
- `@ConditionalOnMissingBean`
- configuration properties
- profiles
- application startup
- embedded server
- Actuator
- custom configuration
- overriding Boot defaults
- debugging auto-configuration

Current 2026 interview material specifically emphasizes dependency injection, IoC, auto-configuration, starters, configuration/profiles, and understanding what happens internally when dependencies are added.

Part C — Spring MVC + REST

Q61–85

- DispatcherServlet
- Controller → Service → Repository
- `@RequestMapping`
- `@GetMapping`
- `@PostMapping`
- `@PathVariable`
- `@RequestParam`
- `@RequestBody`

- DTOs
- validation
- `@Valid`
- global exception handling
- `@ControllerAdvice`
- `ResponseEntity`
- HTTP status codes
- idempotency
- pagination
- API versioning

Part D — Production Spring Boot

Q86–105

- Transactions
- `@Transactional`
- propagation
- isolation
- rollback
- proxy behavior
- self-invocation
- transaction boundaries
- N+1
- connection pools
- caching
- logging
- Actuator
- health checks
- configuration/security mistakes

Part E — Actual Interview Scenarios

Q106–120

These will be the **most valuable**.

For example:

> *"Your API is suddenly taking 5 seconds instead of 200 ms. How do you investigate?"*
>
> *"You added @Transactional, but rollback isn't happening. Why?"*

> *"Two beans implement the same interface. What happens?"*
>
> *"Your application starts locally but fails in production. How do you debug it?"*
>
> *"Why does adding spring-boot-starter-data-jpa suddenly create database-related infrastructure?"*
>
> *"Your API returns 500 for invalid input. How would you redesign the exception handling?"*
>
> *"Your Spring Boot service has 10,000 DB queries for one API request. What's happening?"*

These are the questions that turn **memorization into interview performance**.

Recent 2-year interview guidance specifically recommends being able to build a small end-to-end feature—**controller → service → repository → DTO → validation → exception handling**—and explain the design decisions, rather than only knowing annotation definitions.

chatgpt response: # Module 2

Module 2 — Spring & Spring Boot

Part E — Actual Product-Company Interview Scenarios

Q106–Q120

This is the **most important part of Module 2**.

The first 105 questions taught you the concepts. Here the interviewer gives you a problem and checks whether you can **reason through the system**.

A strong candidate does not immediately jump to:

> *"I'll add @Transactional."*

or:

> *"I'll increase the connection pool."*

Instead:

```text
 Problem
    ↓
 Observe
    ↓
 Measure
    ↓
```

```
Localize
   ↓
Identify root cause
   ↓
Fix
   ↓
Verify
```

That is the mindset expected in real backend interviews.

Q106. Your API suddenly takes 5 seconds instead of 200 ms. How do you investigate?

□□□ **One of the highest-value production questions.**

⭐ Short interview answer

> *"I would first confirm the latency increase using metrics such as p95 and p99 rather than only the average. Then I'd use logs, traces, application metrics, database metrics, and external-service metrics to localize where the additional latency is coming from. I'd check for slow SQL, N+1 queries, connection-pool exhaustion, external API latency, thread or CPU saturation, GC issues, and cache degradation. After identifying the bottleneck, I'd fix the root cause and verify the improvement with the same metrics."*

That is already a good answer.

But the interviewer will probably continue.

Interviewer:

> *"What exactly would you check?"*

Step 1 — Confirm the problem

Suppose:

```text
Before:
p50 = 120 ms
p95 = 200 ms
p99 = 350 ms
```

Now:

```text
p50 = 900 ms
```

```
p95 = 4.2 sec
p99 = 5.8 sec
```

This tells you the problem is widespread rather than just a few outliers.

Step 2 — Look at distributed tracing

Suppose the trace says:

```
text
Controller               5 ms
Business logic          20 ms
Database                4.7 sec
Serialization            5 ms
```

Now the investigation changes.

You don't need to waste time investigating Jackson or controller mapping.

The database is the suspected bottleneck.

Step 3 — Inspect database behavior

Check:

```
text
Query duration
Query count
Connection acquisition time
Lock waits
Execution plan
Database CPU
Database I/O
```

Suppose you discover:

```
text
API request
    ↓
1,001 SQL queries
```

Now you have a strong N+1 hypothesis.

Step 4 — Check connection pool

Suppose:

```
text
 Pool size = 20


 Active = 20
 Idle = 0
 Pending = 100
```

Now requests are waiting for connections.

Potential chain:

```
text
 Slow query
    ↓
 Connection held longer
    ↓
 Pool fills
    ↓
 Requests wait
    ↓
 API latency increases
```

Step 5 — Check external APIs

Maybe:

```
text
 Payment API:
 200 ms → 4.5 sec
```

Then your API is not intrinsically slow; it is waiting on a dependency.

Step 6 — Check JVM/application

Look at:

```
text
 CPU
 Heap
 GC pauses
 Thread pools
 Blocked threads
 Memory pressure
```

For example:

```
text
 CPU = 99%
```

could completely change the diagnosis.

□ Tricky interviewer counter-question

> *"Would you immediately increase the connection pool?"*

✕Bad answer

> *Yes, because more connections mean more throughput.*

⭑ Strong answer

> *"Not immediately. First I'd determine whether the pool is actually the bottleneck. Increasing pool size can hide or worsen the underlying problem, especially if the database is already saturated. I'd look at connection wait time, active connections, query latency, database capacity, and request concurrency before changing the pool."*

Golden production answer

```
text
 Measure
   ↓
 Trace
   ↓
 Application?
 Database?
 External dependency?
   ↓
 Find bottleneck
   ↓
 Fix root cause
   ↓
 Measure again
```

Q107. You added `@Transactional`, but rollback isn't happening. Why?

□□□ **Classic Spring interview question.**

⭑ Short interview answer

> *"I'd first check whether the method invocation is going through the Spring proxy, whether this is self-invocation, whether the exception matches the rollback rules, whether the exception was swallowed, whether the object was created by Spring, and whether the correct transaction manager is being used."*

Spring's default proxy-based transaction mode intercepts external calls through the proxy; self-invocation bypasses that proxy. (Home)

Case 1 — Self-invocation

```java
@Service
public class OrderService {

    public void placeOrder() {
        saveOrder();
    }

    @Transactional
    public void saveOrder() {
        repository.save(...);

        throw new RuntimeException();
    }
}
```

The actual call is effectively:

```java
this.saveOrder();
```

not:

```text
proxy → saveOrder()
```

Therefore the transactional interceptor can be bypassed. (Home)

Case 2 — Checked exception

```java
@Transactional
public void saveOrder() throws Exception {

```

```
    repository.save(...);

    throw new Exception();
}
```

By default, Spring rollback rules primarily roll back on `RuntimeException` and `Error`, not arbitrary checked exceptions. (Home)

Solution:

```
java
@Transactional(
    rollbackFor = Exception.class
)
public void saveOrder() throws Exception {
    ...
}
```

Case 3 — Exception swallowed

```
java
@Transactional
public void saveOrder() {

    try {
        repository.save(...);

        throw new RuntimeException();

    } catch (RuntimeException ex) {
        log.error("Failed", ex);
    }
}
```

Now the transactional interceptor may see normal completion rather than the exception escaping the method.

That can result in a commit when you expected rollback.

Case 4 — Object created manually

```java
OrderService service = new OrderService(...);
service.saveOrder();
```

You bypass Spring's managed proxy.

So:

```text
new OrderService()
      ↓
plain object
      ↓
no Spring transactional interception
```

Case 5 — Wrong transaction boundary

Maybe:

```text
Controller
   ↓
@Transactional method A
   ↓
method B
```

and you're assuming B has its own transaction when actually it is participating in A's transaction.

Understand the **actual boundary**, not merely where the annotation appears.

□ Best debugging sequence

```text
@Transactional
     ↓
Is bean Spring-managed?
     ↓
Is call going through proxy?
     ↓
Self invocation?
     ↓
Exception type?
     ↓
```

```
rollbackFor / noRollbackFor?
     ↓
Exception swallowed?
     ↓
Correct transaction manager?
     ↓
Actual database transaction?
```

That is a very strong interview answer.

Q108. Why doesn't `@Transactional` work during self-invocation?

⭐ Short answer

> *"Because Spring's default transaction advice is proxy-based. An external call goes through the proxy, but a method calling another method on this directly bypasses the proxy, so the transactional interceptor doesn't get a chance to run."*

Current Spring documentation states this explicitly. (Home)

Visual explanation

External call:

```text
Caller
  ↓
Proxy
  ↓
@Transactional advice
  ↓
Target.saveOrder()
```

Self-call:

```text
Target.placeOrder()
      |
      ↓
this.saveOrder()
      |
      ↓
Target.saveOrder()
```

Notice:

```
text
 NO PROXY
```

How should you fix it?

⭐ Best solution — refactor

```
text
 OrderService
      |
      ↓
 OrderPersistenceService
      |
      ↓
 @Transactional
```

Now:

```
text
 OrderService
    ↓
 Spring proxy
    ↓
 OrderPersistenceService
    ↓
 @Transactional
```

Spring's documentation recommends avoiding self-invocation where possible; alternatives include self-injection, while `AopContext.currentProxy()` is specifically discouraged because it couples application code to Spring AOP. (Home)

Q109. Two beans implement the same interface. What happens?

Suppose:

```
java
 public interface PaymentService {
     void pay();
 }
```

Two beans:

```java
 @Service
 public class CardPaymentService
         implements PaymentService {
 }
```

```java
 @Service
 public class UpiPaymentService
         implements PaymentService {
 }
```

And:

```java
 @Service
 public class OrderService {

     private final PaymentService paymentService;

     public OrderService(
             PaymentService paymentService) {
         this.paymentService = paymentService;
     }
 }
```

⭐ Short interview answer

> *"Spring finds multiple candidates for the PaymentService dependency, so the injection is ambiguous. Typically the application fails with a NoUniqueBeanDefinitionException. I can resolve it with @Primary, @Qualifier, or a design that avoids the ambiguity."*

Solution 1 — `@Primary`

```java
 @Primary
 @Service
 public class CardPaymentService
         implements PaymentService {
 }
```

Now:

```text
 PaymentService
       |
  +----+----+
  |         |
 Card      UPI
  ↑
 Primary
```

Solution 2 — `@Qualifier`

```java
 public OrderService(
     @Qualifier("upiPaymentService")
     PaymentService paymentService) {

     this.paymentService = paymentService;
 }
```

Now the choice is explicit.

□ Counter-question

> *"When would you prefer @Qualifier over @Primary?"*

⭐ Answer

> *"I'd use @Primary when one implementation should be the default across most injection points. I'd use @Qualifier when a specific consumer needs a specific implementation."*

Q110. Your application works locally but fails in production. How do you debug it?

□□□

⭐ Short interview answer

> *"I would compare the effective runtime environments rather than assuming the code is different. I'd check active profiles, configuration sources, environment variables, secrets, database connectivity, external services, dependency/runtime versions, network configuration, resource limits, and startup logs. For Spring Boot specifically, I'd inspect bean creation failures, configuration binding, health endpoints, and auto-configuration conditions."*

My debugging order

```text
 1. Exact production error
         ↓
 2. Full stack trace
         ↓
 3. Active profile
         ↓
 4. Effective configuration
         ↓
 5. Environment variables
         ↓
 6. Secrets
         ↓
 7. Database connectivity
         ↓
 8. External dependencies
         ↓
 9. Runtime/JDK/container differences
         ↓
 10. Resource limits
```

Example

Local:

```properties
 spring.datasource.url=jdbc:mysql://localhost/orders
```

Production:

```text
 DB_HOST = orders-db.internal
```

But application expects:

```text
 DATABASE_HOST
```

Result:

```text
 production startup
```

```
        ↓
 missing property
        ↓
 DataSource creation fails
        ↓
 application doesn't start
```

Don't fix this by randomly adding annotations.

Find the configuration mismatch.

Q110.1 Another classic production issue

Local:

```
text
 Profile = dev
```

Production:

```
text
 Profile = prod
```

Then:

```
text
 application-dev.properties
       ↓
 works


 application-prod.properties
       ↓
 wrong property
```

Your code is identical.

Your configuration isn't.

Q111. Why does adding `spring-boot-starter-data-jpa` suddenly create database-related infrastructure?

□□□ This tests whether you understand Part B.

⭐ Strong answer

> *"The starter adds JPA-related dependencies to the classpath. During application startup, Spring Boot discovers applicable auto-configuration candidates and evaluates their conditions. If the required classes and other conditions are present, Boot configures JPA-related infrastructure. Existing user-defined beans can cause Boot's auto-configuration to back off."*

Conceptually:

```text
 Add starter
    ↓
 Dependencies added
    ↓
 Classes appear on classpath
    ↓
 Auto-configuration candidate
    ↓
 Conditions evaluated
    ↓
 Conditions match
    ↓
 JPA infrastructure configured
```

🔁 Follow-up

> *"Does the starter itself create the beans?"*

Answer

> *"No. The starter primarily provides dependencies. Auto-configuration is what interprets the available classpath and conditionally configures the application's infrastructure."*

That's an important distinction.

Another follow-up

> *"Does this automatically mean a production database connection will work?"*

Answer

> *"No. The application still needs an appropriate database driver and valid datasource configuration, unless Boot can derive a supported configuration from the environment."*

Q112. Your API returns 500 for invalid input. How would you redesign exception handling?

□□□

Suppose request:

```
json
 {
   "name": "",
   "email": "wrong"
 }
```

and API returns:

```
http
 500 Internal Server Error
```

That's poor API behavior.

⭐ Strong answer

> *"I'd separate validation failures from unexpected server failures. Request DTOs would contain Bean Validation constraints, the controller would use @Valid, and a global exception handler would translate validation exceptions into a consistent 4xx response. Unexpected exceptions would remain 5xx errors and be logged with a correlation/trace ID without exposing internal details to the client."*

Example DTO

```
java
 public class CreateUserRequest {


     @NotBlank
     private String name;


     @Email
     @NotBlank
     private String email;
 }
```

Controller:

```
java
 @PostMapping
```

```
public ResponseEntity<UserResponse> create(
        @Valid @RequestBody
        CreateUserRequest request) {

    ...
}
```

Global handler:

```
java
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> handleValidation(
            MethodArgumentNotValidException ex) {

        return ResponseEntity
                .badRequest()
                .body(...);
    }
}
```

Response:

```
json
{
  "status": 400,
  "code": "VALIDATION_ERROR",
  "message": "Invalid request",
  "errors": {
    "email": "must be a valid email",
    "name": "must not be blank"
  }
}
```

Q112.1 Why not simply catch `Exception` in every controller?

Because then:

```text
 Controller A → try/catch
 Controller B → try/catch
 Controller C → try/catch
 Controller D → try/catch
```

you get:

```text
 duplicated logic
 inconsistent responses
 harder maintenance
```

Better:

```text
 All Controllers
       ↓
 @RestControllerAdvice
       ↓
 Consistent error handling
```

Q113. Your Spring Boot service makes 10,000 DB queries for one API request. What's happening?

□□□□ **Very common JPA interview scenario.**

⭐ Short answer

> *"My first suspicion would be an N+1 query problem, but I wouldn't assume it. I'd inspect SQL logs, Hibernate statistics or APM traces, count queries per request, identify which entity access triggers repeated SQL, and then determine whether the correct solution is a fetch join, entity graph, projection, batching, or a different query design."*

Example

Suppose:

```java
 List<Order> orders = repository.findAll();


 for (Order order : orders) {
```

```
    order.getItems().size();
}
```

You could get:

```text
1 × orders
+
N × items
```

For:

```text
N = 9999
```

you get:

```text
10,000 queries
```

Fix options

Option 1 — Fetch join

```jpql
SELECT o
FROM Order o
JOIN FETCH o.items
```

Option 2 — Entity graph

```java
@EntityGraph(attributePaths = "items")
List<Order> findAll();
```

Option 3 — DTO projection

Return only the fields the API needs.

Option 4 — Batch fetching

Reduce repeated round trips.

□ Counter-question

> *"Why not make the relationship EAGER?"*

⭐ Answer

> *"Because EAGER is not a universal N+1 solution. It can cause unnecessary data loading, larger joins or result sets, higher memory consumption, and performance problems in other use cases. I'd fetch the data required for the specific use case."*

That's a much stronger answer.

Q114. Your connection pool is exhausted. What could be causing it?

□□□

⭐ Short answer

> *"I'd determine whether connections are being held for too long, whether queries are slow, whether transaction boundaries are too large, whether concurrency exceeds database capacity, whether there is connection leakage, or whether the pool is incorrectly sized."*

Typical chain

```text
 Slow SQL
    ↓
 Connection held longer
    ↓
 Pool fills
    ↓
 Requests wait
    ↓
 Latency rises
    ↓
 More requests queue
    ↓
 System becomes unstable
```

Another cause — long transaction

Imagine:

```java
 @Transactional
 public void placeOrder() {


     saveOrder();
```

```
    callSlowExternalService(); // 5 seconds

    updatePayment();
}
```

Now the transaction may remain active while waiting for the external service.

That can increase connection occupancy.

Another cause — pool leak

If application code or custom infrastructure fails to properly return connections, available connections can continually fall.

In normal Spring Data/JPA usage, connection/resource lifecycle is generally handled by the framework, so investigate custom JDBC/resource handling too.

□ Counter-question

> *"Pool is exhausted. Should I double the pool size?"*

⭐ Answer

> *"Only after checking database capacity and why connections are occupied. Increasing the pool may improve throughput if the database has spare capacity, but if slow queries or database saturation are the root cause, a larger pool can make the problem worse."*

Q115. Your cache returns stale data. What would you do?

□□□

Example

Database:

```
text
 User.name = "John Smith"
```

Cache:

```
text
 User.name = "John"
```

API reads:

```text
 Cache HIT
    ↓
 returns John
```

even though DB says:

```text
 John Smith
```

⭐ Strong answer

> *"I'd first identify the caching strategy and ownership of the source of truth. For a cache-aside design, after a successful update I'd usually invalidate or update the corresponding cache entry. I'd also check TTL, cache keys, concurrent updates, multi-instance behavior, and whether there are multiple writers that bypass invalidation."*

Q115.1 Cache-aside flow

Read:

```text
 Request
   ↓
 Cache?
  /   \
 Hit  Miss
  |     |
  ↓     ↓
 Return DB
        ↓
      Cache
        ↓
      Return
```

Write:

```text
 Request
  ↓
 Update DB
```

```
  ↓
Invalidate cache
```

The difficult part is:

> *Keeping cache and database behavior sufficiently consistent for the business requirement.*

Q115.2 What if two application instances use local memory cache?

Instance A:

```
text
Cache A = John
```

Instance B:

```
text
Cache B = John
```

Update through A:

```
text
DB = John Smith
Cache A = invalidated
```

But:

```
text
Cache B = John
```

Now the system becomes inconsistent across instances.

This is one reason distributed caches such as Redis can be used where shared cache state is required.

Q116. Kubernetes keeps restarting your Spring Boot application. How would you investigate?

□□□ Very practical modern interview scenario.

⭐ Strong answer

> *"I'd determine whether the container is actually crashing, being killed because of resource limits, or failing a liveness/readiness probe. I'd inspect pod events, container exit codes, application startup logs, JVM memory/GC metrics, health endpoints, and CPU/memory limits. I'd also check whether a health probe is incorrectly treating an external dependency failure as application liveness."*

Possible causes

```text
Application exception
        ↓
Process exits
```

or:

```text
Memory exceeds limit
        ↓
OOMKilled
```

or:

```text
Liveness probe fails
        ↓
Kubernetes restarts container
```

or:

```text
Startup takes too long
        ↓
Probe misconfigured
        ↓
Container killed
```

Spring Boot's documentation specifically cautions against making liveness depend on external systems because a dependency failure can otherwise cause unnecessary restarts and cascading failures. (Home)

Q116.1 Liveness vs readiness in this scenario

Suppose:

```text
Database temporarily unavailable
```

You might want:

```text
 Liveness = UP
 Readiness = DOWN
```

rather than:

```text
 Liveness = DOWN
```

because the process itself is healthy and should not necessarily be restarted just because a dependency is temporarily unavailable.

Q117. CPU or memory usage suddenly increases. What do you investigate?

⭐ Strong answer

I'd avoid immediately increasing instance size.

First determine whether the increase is:

```text
 CPU
 or
 Memory
```

If CPU is high

Check:

```text
 Hot methods
 Thread activity
 Database query volume
 Serialization
 Infinite loops
 Retry storms
 Excessive logging
 GC
```

For example:

```text
 External API failing
    ↓
```

```
Retry immediately
   ↓
More requests
   ↓
More CPU
   ↓
More failures
   ↓
More retries
```

That's a retry storm.

If memory is high

Check:

```text
Heap usage
GC frequency
Object allocation
Large collections
Caching
Huge API responses
Memory leaks
Unbounded queues
```

Example:

```java
List<User> users = repository.findAll();
```

against millions of rows can create massive memory pressure.

□ Counter-question

> *"What metric would you look at first?"*

There isn't one universal answer.

A strong answer is:

> *"I'd first determine whether the resource pressure is CPU, heap, off-heap/native memory, or another resource, then use the corresponding metrics and profiling/tracing tools."*

Q118. An external payment API sometimes takes 10 seconds. How would you design your Spring Boot service?

□□□ Strong product-company scenario.

Suppose:

```
text
 Order Service
     ↓
 Payment Service
     ↓
 10-second response
```

You should immediately think about:

```
text
 Timeout
 Retry
 Circuit breaker
 Idempotency
 Connection pool
 Transaction boundary
 Observability
 Fallback
```

⭐ Strong answer

> *"I would set an explicit timeout rather than allowing requests to hang indefinitely. I'd retry only when the operation is safe to retry and with controlled backoff. For payment operations I'd use an idempotency key to prevent duplicate charges. I'd avoid holding a database transaction open while waiting on the external service unless the business design genuinely requires it. I'd also monitor latency, timeout rates, and failures and consider circuit-breaking or asynchronous processing depending on the business requirement."*

Q118.1 Why is retry dangerous for payments?

Imagine:

```
text
 POST /payment
```

Server receives it:

```text
 Payment succeeds
```

But response is lost.

Client retries:

```text
 POST /payment
```

Now you could charge twice.

Therefore:

```text
 Retry
 +
 No idempotency
 =
 danger
```

A robust design:

```text
 Idempotency-Key: payment-12345
```

Server:

```text
 First request
     ↓
 process
     ↓
 store result

 Retry
     ↓
 same key
     ↓
 return previous result
```

Q119. Design a production-ready Order API.

□□□ **Major product-company system-design-style Spring question.**

Interviewer:

> *"Design a POST /orders endpoint."*

Don't jump directly into code.

Step 1 — API contract

```http
POST /api/v1/orders
Idempotency-Key: abc123
Content-Type: application/json
```

Request:

```json
{
  "customerId": 101,
  "items": [
    {
      "productId": 500,
      "quantity": 2
    }
  ]
}
```

Step 2 — Controller

```text
POST /orders
      ↓
OrderController
      ↓
@Valid
      ↓
OrderService
```

Controller should not contain the complete business workflow.

Step 3 — Service

```text
 OrderService.createOrder()
        |
        +-- validate customer
        |
        +-- validate products
        |
        +-- calculate price
        |
        +-- create order
        |
        +-- reserve inventory
        |
        +-- payment handling
```

But immediately ask:

> *Which operations must be atomic?*

Step 4 — Transaction

Database operations that must be atomic could be inside:

```java
 @Transactional
 public OrderResponse createOrder(...) {
     ...
 }
```

But don't blindly include an external payment HTTP call in the transaction.

A better design might be:

```text
 DB transaction
    ↓
 create order
    ↓
 store payment request/state
    ↓
 commit
```

Then external payment processing happens through an appropriate synchronous or asynchronous workflow depending on requirements.

Step 5 — Database constraints

For example:

```
text
 order.id PRIMARY KEY
 product.id PRIMARY KEY
 unique(idempotency_key, customer_id)
```

Don't rely only on:

```
java
 if (!exists(...)) {
     insert();
 }
```

because concurrent requests can race.

Step 6 — Response

Successful creation:

```
http
 201 Created

json
 {
   "orderId": 5001,
   "status": "CREATED"
 }
```

Potential duplicate/idempotency behavior should also be clearly defined.

Step 7 — Observability

Track:

```
text
 order creation latency
 payment latency
```

```
DB latency
failure rate
inventory failures
duplicate requests
```

Log:

```
text
traceId
orderId
customerId
```

but don't log:

```
text
payment credentials
access tokens
card data
```

Q119.1 Interviewer asks:

> *"What if two requests create the same order?"*

Your answer should mention:

```
text
Idempotency key
+
Database uniqueness
+
Atomic state transition
```

Q119.2 Interviewer asks:

> *"What if payment succeeds but database update fails?"*

Now you have crossed from simple CRUD into distributed consistency.

A sophisticated answer:

> *"I would not assume a single database transaction can atomically cover an external payment service. I'd model the order/payment state explicitly and use a reliable workflow, potentially with an outbox/event-driven approach and reconciliation depending on the business requirements."*

This is exactly where real system design begins.

Q120. Full Spring Boot debugging interview

□□□□ **Final question of Module 2**

Imagine the interviewer says:

> *"You are on call. Your Spring Boot Order API is experiencing a production incident."*

Facts:

```
text
 Normal latency:
 200 ms

 Current:
 5 seconds

 Error rate:
 8%

 Database CPU:
 85%

 Connection pool:
 100% utilized

 SQL queries/request:
 1,200

 Cache hit rate:
 20% → previously 90%

 One external API:
 8 seconds latency
```

Interviewer:

> *"Walk me through what you do."*

⭐ Excellent answer

I would not start changing code immediately.

Step 1 — Confirm the scope

I'd look at:

```
text
 p50
 p95
 p99
 error rate
 traffic
```

to determine whether this is:

```
text
 all requests
 or
 specific endpoints/users/regions
```

Step 2 — Use traces

I'd identify the latency distribution:

```
text
 Controller       5 ms
 Business logic  30 ms
 DB             3 sec
 External API   1.8 sec
```

Now I know the problem is multi-factor.

Step 3 — Investigate the database

1,200 queries/request is suspicious.

I'd inspect:

```
text
 SQL logs / APM
 query count
 repeated queries
 slow queries
 execution plans
```

If I find:

```
text
 1 + N queries
```

I'd investigate N+1.

Step 4 — Investigate connection pool

Pool at:

```
text
 100% utilization
```

means requests are competing for DB connections.

I'd inspect:

```
text
 active connections
 pending requests
 connection acquisition time
 maximum pool size
 query duration
```

I wouldn't automatically increase the pool.

Step 5 — Investigate cache

Cache hit rate:

```
text
 90% → 20%
```

That is a huge clue.

Possible reasons:

```
text
 Cache evictions
 Redis outage
 Wrong cache key
 TTL change
 Traffic pattern change
 Serialization problem
```

```
Cold cache
Cache connectivity/latency
```

If cache misses are now hitting the database, this could explain:

```
text
Cache hit ↓
   ↓
DB traffic ↑
   ↓
DB CPU ↑
   ↓
Connections occupied ↑
   ↓
Latency ↑
```

Step 6 — Investigate external API

External API:

```
text
8 seconds
```

I'd determine:

```
text
Why are we calling it?
How many times/request?
Is it inside a transaction?
Are requests retrying?
Is timeout configured?
```

Especially dangerous:

```
text
External service slow
    ↓
Retry
    ↓
Retry
    ↓
Retry
```

```
    ↓
More traffic
```

Step 7 — Check transaction boundaries

If an external API is being called inside:

```java
@Transactional
```

I would investigate whether database connections are being held while waiting for that remote response.

That could contribute to pool exhaustion.

Step 8 — Immediate mitigation

Depending on evidence:

```text
Temporarily disable problematic cache behavior
Reduce external API retries
Enable circuit breaker
Rollback recent deployment
Rate-limit traffic
Scale application instances
Reduce expensive endpoint traffic
```

But only after understanding the failure mode.

Step 9 — Root-cause fix

Possible final fixes:

```text
Fix N+1
+
Restore correct caching
+
Correct external-service timeout/retry policy
+
Shorten transaction boundaries
+
Optimize slow SQL
```

Step 10 — Verify

After deploying:

```
text
 p95 ↓
 p99 ↓
 error rate ↓
 SQL queries/request ↓
 DB CPU ↓
 pool utilization ↓
 cache hit rate ↑
 external API impact ↓
```

You don't declare victory simply because the server "looks healthy."

You measure the original problem again.

□ Q120 — What the interviewer is really testing

The interviewer isn't actually testing:

```
text
 Do you know @Transactional?
 Do you know @Cacheable?
 Do you know @RestController?
```

They're testing whether you understand:

```
text
                    REQUEST
                       |
           +-----------+-------------+
           |           |             |
           ↓           ↓             ↓
        Spring      Database    External API
           |           |             |
           ↓           ↓             ↓
      Proxy/AOP     Pool        Timeout
           |           |             |
           ↓           ↓             ↓
    Transaction      SQL        Retry
```

```
          |            |             |
          +------------+-------------+
                       |
                       ↓
                 Observability
                       |
            Logs / Metrics / Traces
```

□ Module 2 — Final Interview Map

You have now completed the full **120-question Spring + Spring Boot interview module**.

```
text
 PART A — Spring Core
 Q1–30
     ↓
 IoC
 DI
 BeanFactory
 ApplicationContext
 Beans
 Lifecycle
 Scopes
 @Autowired
 @Component
 @Service
 @Repository
 @Configuration
 @Bean
 AOP
 Proxy

 PART B — Spring Boot Internals
 Q31–60
     ↓
 Spring vs Boot
 @SpringBootApplication
 Auto-configuration
 Starters
```

```
Conditions
Configuration
Profiles
Startup
Embedded server
Actuator
Debugging Boot

PART C — Spring MVC + REST
Q61–85
    ↓
HTTP request lifecycle
DispatcherServlet
Controller
Service
Repository
DTO
Validation
Exception handling
HTTP status
Idempotency
Pagination
Versioning

PART D — Production Spring Boot
Q86–105
    ↓
Transactions
@Transactional
Propagation
Isolation
Rollback
Proxy behavior
Self invocation
N+1
Connection pool
Caching
Logging
```

```
Health
Security/configuration


PART E — Interview Scenarios
Q106–120
    ↓
Production debugging
Performance
Transactions
Beans
Configuration
JPA
Caching
Kubernetes
External APIs
End-to-end system reasoning
```

□ The 25 Spring questions I would prioritize before a real interview

You do **not** need to give equal study time to all 120.

Master these particularly well:

Spring Core

```
text
1. IoC
2. DI
3. Bean lifecycle
4. BeanFactory vs ApplicationContext
5. Constructor injection
6. @Autowired
7. @Component/@Service/@Repository
8. @Bean vs @Component
9. @Primary vs @Qualifier
10. Spring proxies
```

Spring Boot

```
text
11. @SpringBootApplication
```

```
12. Auto-configuration
13. Starters
14. @ConditionalOnClass
15. @ConditionalOnMissingBean
16. Boot startup lifecycle
17. Configuration/profiles
```

Spring MVC

```
text
18. DispatcherServlet
19. Controller-Service-Repository
20. DTO + validation
21. @ControllerAdvice
22. Request/response lifecycle
```

Production

```
text
23. @Transactional + propagation + rollback
24. Self-invocation/proxy problem
25. N+1 + connection pool + production debugging
```

□ One final interview rule

For almost every Spring question, answer in **three levels**.

Level 1 — Definition

> *"@Transactional provides declarative transaction management."*

Level 2 — Internal working

> *"Spring generally applies it through AOP proxying and a transaction interceptor that coordinates with a transaction manager."*

Spring's current documentation explicitly describes this proxy/interceptor/transaction-manager model. (Home)

Level 3 — Real production consequence

> *"Therefore, self-invocation can bypass the proxy, and an incorrectly chosen transaction boundary can hold database resources longer than necessary."*

That third level is what turns a **2-year candidate from annotation memorization into someone who can actually debug Spring applications**.

✅Module 2 is now complete

Module 2: Spring + Spring Boot — 120 questions

You now have the full progression:

```
text
 Spring basics
       ↓
 Spring internals
       ↓
 Spring Boot internals
       ↓
 HTTP/MVC
       ↓
 REST
       ↓
 Transactions
       ↓
 Database performance
       ↓
 Caching
       ↓
 Production debugging
```

The next module in your original master plan is:

Module 3 — JPA / Hibernate

80 Questions + Answers

The important shift here will be from:

```
text
 Spring Boot
```

to:

```
text
 Spring Data JPA
         ↓
 JPA
```

```
        ↓
Hibernate
        ↓
Persistence Context
        ↓
Entity Lifecycle
        ↓
Dirty Checking
        ↓
First-Level Cache
        ↓
Second-Level Cache
        ↓
Lazy/Eager Loading
        ↓
N+1
        ↓
JPQL / Criteria / Native Query
        ↓
Fetch Join
        ↓
Optimistic/Pessimistic Locking
        ↓
Transactions + JPA
```

And for the JPA module, the particularly high-value questions will be **Persistence Context, Entity Lifecycle, dirty checking,** `save()` **internals,** `flush()` **vs** `commit()`**,** `findById()` **vs** `getReferenceById()`**, LAZY vs EAGER, N+1,** `JOIN FETCH`**, cascading, orphan removal, optimistic locking, and why Hibernate generates unexpected SQL**.